Econometrics Assignment

Luca Girlando - Irene Balan

May 2025

Contents

I Theoretical Exercises **3**

1 T1 ... **3**

1.1 Exercise 1: A first class of estimators ... 3
1.1.1 1.a) Unbiasedness proof ... 3
1.1.2 1.b) Variance and covariance computation ... 4
1.1.3 1.c) Optimal estimator choice ... 4

1.2 Exercise 2: A linear combination of estimators ... 5
1.2.1 2.a) Expectation and variance of b^* ... 5
1.2.2 2.b) Unbiasedness Condition for the Combined Estimator b^* ... 5
1.2.3 2.c) Derivation of the Best Linear Unbiased Estimator (BLUE) ... 6

1.3 Exercise 3: The multivariate regression model ... 8
1.3.1 3.a) Model specification in matrix notation ... 8
1.3.2 3.b) OLS estimator properties ... 8
1.3.3 3.c) Regression results analysis ... 9
1.3.4 3.d) Joint Hypothesis Testing: Restricted Model Analysis ... 11

2 T2 ... **13**

2.1 Exercise 1: Preliminary moments ... 13

2.2 Exercise 2: Contemporaneous covariance between Xt and ut ... 13

2.3 Exercise 3: Algebraic decomposition of the OLS slope estimator ... 14

2.4 Exercise 4: Slutsky-type approximation as $T \to \infty$ ... 15

2.5 Exercise 5: Large-sample variance of $\hat{\beta}_1$ ... 15

3 T3 ... **17**

3.1 Exercise 1: ARCH model for the variance ... 17

3.2 Exercise 2: GARCH model for the variance ... 18

3.3 Exercise 3: Numerical illustration ... 18

II Empirical Applications **20**

4 E1 ..... **20**

4.1 Returns of S&P500 index and companies 20
4.1.1 Point 1: Daily Returns Calculation and Initial Data 20
4.1.2 Point 2: S&P500 Statistical Properties and Volatility Dynamics 20
4.1.3 Point 3: Individual Stock Return Distributions 23
4.1.4 Point 4: Recent One-Year Performance Examination 24
4.1.5 Point 5: Full Sample vs Recent Period Comparison 26
4.1.6 Point 6: Monthly Returns Computation and Analysis 28

4.2 Treasury Bill 29

4.3 Fama-French Portfolios and factors 30
4.3.1 Point 7: Portfolio Performance Analysis 30
4.3.2 Point 8: Factor Correlation Analysis 31

5 E2 ..... **33**

5.1 Point 1: CAPM Beta Estimation and Analysis 33

5.2 Point 2: Naïve Time-Series Test of the CAPM 36

6 E3 ..... **40**

6.1 Point 1: CAPM Cross-Sectional Test Results 40

6.2 Point 2: Fama-French Portfolios CAPM Test 41

6.3 Point 3: Comparative Analysis of Risk Premia 43

7 E4 ..... **45**

7.1 Model Estimation and Interpretation 45

7.2 Residual Diagnostics 46

7.3 ACF and PACF Analysis 50

7.4 Conclusion 50

III Appendix **51**

8 Matlab **51**

8.1 Codes 51

8.2 Functions 77

Part I
Theoretical Exercises

1 T1

1.1 Exercise 1: A first class of estimators

1.1.1 1.a) Unbiasedness proof

Let us thoroughly examine the statistical properties of these single-observation estimators. The unbiasedness property is fundamental in estimation theory, ensuring that our estimators correctly target the parameter of interest on average.

We begin with a set of N distinct estimators defined as:

$$\hat{b_t} = \frac{y_t}{x_t}, \quad \text{for } t = 1, \ldots, N$$

Each estimator uses a single data point (x_t, y_t), making this a particularly simple but illuminating case study in estimation theory.

These estimators derive from the classical linear regression model:

$$y_t = bx_t + \varepsilon_t \quad \text{for } t = 1, \ldots, N$$

where the components have precise statistical interpretations:

- y_t: The dependent variable (market capitalization), representing the outcome we wish to model
- x_t: The independent variable (trading volume), treated as fixed and known
- ε_t: The random disturbance term with $\mathbb{E}[\varepsilon_t] = 0$, capturing all other factors affecting y_t
- b: The true population parameter we wish to estimate

To prove unbiasedness, we perform a careful decomposition:

$$\hat{b_t} = \frac{y_t}{x_t} = \frac{bx_t + \varepsilon_t}{x_t} = b + \frac{\varepsilon_t}{x_t}$$

Taking expectations and applying the linearity property:

$$\mathbb{E}[\hat{b_t}] = \mathbb{E}\left[b + \frac{\varepsilon_t}{x_t}\right] = b + \frac{1}{x_t}\mathbb{E}[\varepsilon_t] = b$$

This derivation establishes that:

- The expectation of each estimator equals the true parameter b
- The unbiasedness holds regardless of the specific x_t value
- The result depends crucially on the zero-mean error assumption

1.1.2 1.b) Variance and covariance computation

Beyond unbiasedness, we must examine the precision and dependence structure of these estimators. The variance calculation reveals:

$$\text{Var}(\hat{b_t}) = \text{Var}\left(b + \frac{\varepsilon_t}{x_t}\right) = \frac{\text{Var}(\varepsilon_t)}{x_t^2} = \frac{\sigma^2}{x_t^2}$$

Key observations from this result:

- Precision improves quadratically with x_t magnitude
- The variance depends on both the error variance σ^2 and the regressor value
- Larger trading volumes yield substantially more precise estimates

For the covariance structure between any two estimators ($t \neq s$):

$$\text{Cov}(\hat{b_t}, \hat{b_s}) = \text{Cov}\left(\frac{\varepsilon_t}{x_t}, \frac{\varepsilon_s}{x_s}\right) = \frac{1}{x_t x_s}\text{Cov}(\varepsilon_t, \varepsilon_s) = 0$$

The zero covariance indicates:

- Estimators from different observations are uncorrelated
- This follows from our assumption of independent errors
- In financial applications, this might require checking for time-series dependence

1.1.3 1.c) Optimal estimator choice

Given the variance properties, we can establish an optimal selection criterion. Since:

$$\text{Var}(\hat{b_t}) = \frac{\sigma^2}{x_t^2}$$

we deduce that:

- The minimal variance occurs at the maximal x_t value
- Let $x_{(N)} = \max\{x_1, ..., x_N\}$ be the largest volume
- The optimal single-observation estimator is $\hat{b}_{(N)} = y_{(N)}/x_{(N)}$

This optimality has important implications:

- In practice, we should prefer observations with largest trading volumes
- The efficiency gain can be substantial when x_t values vary significantly
- This provides a simple rule for data selection in this context

1.2 Exercise 2: A linear combination of estimators

1.2.1 2.a) Expectation and variance of b^*

We now consider combining all available data through a weighted average of estimators:

$$b^* = \sum_{t=1}^{N} c_t \hat{b_t} = \sum_{t=1}^{N} c_t \left(\frac{y_t}{x_t} \right)$$

Substituting the model equation:

$$b^* = \sum_{t=1}^{N} c_t \left(b + \frac{\varepsilon_t}{x_t} \right) = b \sum_{t=1}^{N} c_t + \sum_{t=1}^{N} \frac{c_t}{x_t} \varepsilon_t$$

The expectation calculation reveals:

$$\mathbb{E}[b^*] = b \sum_{t=1}^{N} c_t + \sum_{t=1}^{N} \frac{c_t}{x_t} \underbrace{\mathbb{E}[\varepsilon_t]}_{0} = b \sum_{t=1}^{N} c_t$$

For the variance, using error independence:

$$\text{Var}(b^*) = \text{Var} \left(\sum_{t=1}^{N} \frac{c_t}{x_t} \varepsilon_t \right) = \sum_{t=1}^{N} \left(\frac{c_t}{x_t} \right)^2 \sigma^2 = \sigma^2 \sum_{t=1}^{N} \frac{c_t^2}{x_t^2}$$

1.2.2 2.b) Unbiasedness Condition for the Combined Estimator b^*

To derive the condition under which the combined estimator b^* is unbiased, we begin with the following assumptions:

- Each individual estimator b_t is unbiased: $\mathbb{E}(b_t) = \beta$ for all $t = 1, \ldots, N$
- This unbiasedness stems from the zero-mean error assumption: $\mathbb{E}(\varepsilon_t) = 0$

Consider the linear combination of estimators:

$$b^* = \sum_{t=1}^{N} c_t b_t$$

where c_t are scalar weights assigned to each estimator.

Applying the linearity of expectations:

$$\mathbb{E}(b^*) = \mathbb{E} \left(\sum_{t=1}^{N} c_t b_t \right) = \sum_{t=1}^{N} c_t \mathbb{E}(b_t) = \sum_{t=1}^{N} c_t \beta = \beta \sum_{t=1}^{N} c_t$$

For b^* to be unbiased, we require $\mathbb{E}(b^*) = \beta$, which leads to the fundamental condition:

$$\sum_{t=1}^{N} c_t = 1$$

This constraint has several important implications:

- **Parameter Targeting**: Ensures the combined estimator correctly estimates the true parameter β
- **Optimization Requirement**: Must be enforced in any weight optimization procedure
- **Geometric Interpretation**: Defines a hyperplane in the N-dimensional weight space
- **Scale Invariance**: Maintains unbiasedness regardless of β's magnitude

The condition represents a linear restriction on the weights that must be satisfied while maintaining other desirable properties of the combined estimator (e.g., minimal variance under the Gauss-Markov assumptions).

1.2.3 2.c) Derivation of the Best Linear Unbiased Estimator (BLUE)

The Optimization Framework We seek the minimum-variance linear combination of unbiased estimators while maintaining unbiasedness. The general optimization problem is:

$$\min_{\{c_t\}_{t=1}^N} \mathbb{V}\text{ar}\left(\sum_{t=1}^N c_t b_t\right) \quad \text{subject to} \quad \sum_{t=1}^N c_t = 1$$

For estimators with variances $\mathbb{V}\text{ar}(b_t) = \sigma_t^2$, this becomes:

$$\min_{\{c_t\}} \sum_{t=1}^N c_t^2 \sigma_t^2 \quad \text{s.t.} \quad \sum_{t=1}^N c_t = 1$$

Solution for the Two-Estimator Case ($N = 2$) The constrained optimization yields explicit solutions when $N = 2$:

1. Express b^* in terms of a single weight:
$$b^* = c_1 b_1 + (1 - c_1) b_2$$
2. Compute the variance:
$$\mathbb{V}\text{ar}(b^*) = c_1^2 \sigma_1^2 + (1 - c_1)^2 \sigma_2^2$$
3. First-order condition for optimality:
$$\frac{d}{dc_1} \mathbb{V}\text{ar}(b^*) = 2c_1 \sigma_1^2 - 2(1 - c_1)\sigma_2^2 = 0$$
4. Optimal weights emerge as:
$$c_1^{**} = \frac{\sigma_2^2}{\sigma_1^2 + \sigma_2^2}, \quad c_2^{**} = \frac{\sigma_1^2}{\sigma_1^2 + \sigma_2^2}$$

General Solution for N Estimators The solution generalizes through the method of Lagrange multipliers:

$$c_t^{**} = \frac{1/\sigma_t^2}{\sum_{s=1}^{N} 1/\sigma_s^2} = \frac{\sigma_t^{-2}}{\sum_{s=1}^{N} \sigma_s^{-2}}$$

The BLUE estimator takes the form:

$$b^{**} = \sum_{t=1}^{N} \left(\frac{\sigma_t^{-2}}{\sum_{s=1}^{N} \sigma_s^{-2}} \right) b_t$$

Optimality Properties and Connections The BLUE estimator possesses several key characteristics:

- **Minimum Variance**: Achieves the Cramér-Rao lower bound among linear estimators
- **Efficiency**: Implements inverse-variance weighting, giving more weight to precise estimates
- **Equivalence to WLS**: Mathematically identical to weighted least squares under heteroskedasticity
- **Consistency**: Maintains consistency under standard regularity conditions
- **Gauss-Markov Implementation**: Satisfies all Gauss-Markov theorem requirements

Connection to OLS When $\sigma_t^2 = \sigma^2$ (homoskedastic case), the estimator reduces to:

$$b^{**} = \frac{1}{N} \sum_{t=1}^{N} b_t$$

This demonstrates that OLS is a special case of BLUE when errors are homoskedastic and uncorrelated.

Theoretical Justification The optimal weights emerge from:

- Variance decomposition properties
- Quadratic optimization with linear constraints
- First-order conditions from Lagrangian optimization
- Information matrix considerations

1.3 Exercise 3: The multivariate regression model

1.3.1 3.a) Model specification in matrix notation

The multivariate regression framework provides a powerful tool for analyzing relationships between multiple economic variables. The matrix representation offers both computational efficiency and theoretical clarity:

$$y = X\beta + \varepsilon$$

Where the components have precise econometric interpretations:

- y: $n \times 1$ vector of dependent variables (market returns)
- X: $n \times 3$ design matrix containing:
  - First column: Unit vector for the intercept (a)
  - Second column: Market volatility measures (z_t)
  - Third column: Economic policy uncertainty index (q_t)
- β: 3×1 parameter vector $(b_1, b_2, b_3)'$
- ε: $n \times 1$ vector of disturbance terms

The expanded matrix form reveals the complete structure:

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_w \end{bmatrix} = \begin{bmatrix} 1 & z_1 & q_1 \\ 1 & z_2 & q_2 \\ \vdots & \vdots & \vdots \\ 1 & z_w & q_w \end{bmatrix} \begin{bmatrix} \beta_1 \\ \beta_2 \\ \beta_3 \end{bmatrix} + \begin{bmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_w \end{bmatrix}$$

1.3.2 3.b) OLS estimator properties

The Ordinary Least Squares estimator has important statistical properties:

$$\hat{\beta} = (X'X)^{-1}X'y = Ay \quad \text{where} \quad A = (X'X)^{-1}X'$$

The dimensions of each component:

- X: $n \times 3$ design matrix
- X': $3 \times n$ transpose matrix
- $(X'X)^{-1}$: 3×3 inverse matrix

Unbiasedness proof:

$$\mathbb{E}[\hat{\beta}] = \mathbb{E}[(X'X)^{-1}X'y] = (X'X)^{-1}X'\mathbb{E}[y] = (X'X)^{-1}X'X\beta = I_3\beta = \beta$$

1.3.3 3.c) Regression results analysis

Variable	Coefficient	Std.Error	t-stat	P-value
Intercept	22.5569	1.0370	21.7529	0.0187
β_2		0.0961	18.7615	0.0000
β_3	-2.4003	0.0813	-29.5384	0.0000
β_4	-2.4464	0.0924		0.0000
R-squared	0.9765		Sum squared residual	6.837
Adj R-squared	0.9740		Durbin-Watson stat	2.0919
Var-Cov Matrix	Intercept	β_2	β_3	β_4
Intercept	1.07529	-0.00093	-0.04206	-0.0302
β_2			-0.00456	-0.0020
β_3			0.00660	0.00098
β_4				0.00854

Figure 1: Regression analysis results (1984-2024)

3.c.i) Completing the table

$$\hat{\beta}_2 = 0.0961 \times 18.7615 = 1.8029$$

$$t_{\beta_4} = \frac{-2.4464}{0.0924} = -26.4762$$

$$\text{Var}(\beta_2) = (0.0961)^2 = 0.00923$$

Variable	Coefficient	Std.Error	t-stat	P-value
Intercept	22.5569	1.0370	21.7529	0.0187
β_2	***1.8029***	0.0961	18.7615	0.0000
β_3	-2.4003	0.0813	-29.5384	0.0000
β_4	-2.4464	0.0924	***-26.4762***	0.0000
R-squared	0.9765		Sum squared residual	6.837
Adj R-squared	0.9740		Durbin-Watson stat	2.0919
Var-Cov Matrix	Intercept	β_2	β_3	β_4
Intercept	1.07529	-0.00093	-0.04206	-0.0302
β_2	***-0.00093***	***0.00923***	-0.00456	-0.0020
β_3	***-0.04206***	***-0.00456***	0.00660	0.00098
β_4	***-0.0302***	***-0.0020***	***0.00098***	0.00854

Figure 2: Complete regression analysis results (1984-2024)

3.c.ii) Statistical and Economic Interpretation of Coefficients

- **Market Volatility (β_2)**:
  - Estimated coefficient: 1.8029 (standard error: 0.0961)
  - Economic significance: A one-unit increase in market volatility leads to a 1.8029 unit increase in the dependent variable

- Statistical significance: $t = 18.76$ ($p < 0.001$), indicating strong evidence against the null hypothesis of no effect
- 95% Confidence Interval: $[1.607, 1.999]$

- **Coefficient** β_4:
  - Estimated coefficient shows strong negative relationship ($t = -26.4762$)
  - The magnitude suggests substantial economic impact
  - Extremely statistically significant ($p < 0.0001$)
  - Interpretation consistent with theoretical expectations

3.c.iii) Formal Hypothesis Testing Procedures

- **Test of** $\beta_2 = 1$:

$$t = \frac{\hat{\beta}_2 - \beta_2^0}{SE(\hat{\beta}_2)} = \frac{1.8029 - 1}{0.0961} = 8.3548$$

  - Critical value ($\alpha = 0.05$, $df = 36$): $t_{crit} = 1.684$
  - Decision: Reject H_0 ($p < 0.0001$)
  - Conclusion: The effect of market volatility is statistically greater than 1

- **Test of equality** $\beta_3 = \beta_4$:

$$t = \frac{\hat{\beta}_3 - \hat{\beta}_4}{\sqrt{Var(\hat{\beta}_3) + Var(\hat{\beta}_4) - 2Cov(\hat{\beta}_3, \hat{\beta}_4)}}$$

$$t = \frac{-2.4003 - (-2.4464)}{\sqrt{0.00660 + 0.00854 - 2 \times 0.00098}} = 0.401$$

  - Critical value remains $t_{crit} = 1.684$
  - Decision: Fail to reject H_0 ($p = 0.691$)
  - Conclusion: No statistical evidence of difference between β_3 and β_4

3.c.iv) Analysis of Variance Decomposition The goodness-of-fit measures reveal:

$$\begin{aligned}
\text{Total Variation (TSS)} &= \frac{RSS}{1 - R^2} = \frac{6.837}{1 - 0.9765} = 290.936 \\
\text{Explained Variation (ESS)} &= TSS - RSS = 290.936 - 6.837 = 284.099 \\
\text{Unexplained Variation (RSS)} &= 6.837 \quad \text{(Given)}
\end{aligned}$$

- High R^2 (0.9765) indicates the model explains 97.65% of total variation
- Large ESS relative to RSS suggests strong explanatory power

3.c.v) Autocorrelation Diagnostics

- **Durbin-Watson Test**:

$$DW = 2.0919 \quad \text{(Close to ideal value of 2)}$$

  - Formal hypothesis test:
    - H_0: No first-order autocorrelation ($\rho = 0$)
    - H_1: First-order autocorrelation exists ($\rho \neq 0$)
  - Decision: Fail to reject H_0 at $\alpha = 0.05$
  - Conclusion: No evidence of autocorrelation in residuals

- **Implications**:
  - OLS standard errors remain valid
  - No need for autocorrelation-robust standard errors
  - Model specification appears adequate regarding time dependence

3.c.vi) Joint Significance of Slope Parameters The intercept automatically ensures:

$$\mathbb{E}[\epsilon] = 0$$

by construction in OLS estimation. Therefore:

- No need for empirical test of error term mean
- Theoretical guarantee from inclusion of constant term
- Implication: Residuals will always sum to zero in models with intercept

1.3.4 3.d) Joint Hypothesis Testing: Restricted Model Analysis

3.d.i) Restricted Model Specification Originally, the unrestricted model is:

$$y_t = \beta_1 + \beta_2 X_{2t} + \beta_3 X_{3t} + \beta_4 X_{4t} + \varepsilon_t$$

Under the null hypothesis $H_0 : \beta_2 = 1$ and $\beta_3 = \beta_4$, the restricted model becomes:

$$y_t = \beta_1 + X_{2t} + \gamma_2 (X_{3t} + X_{4t}) + \varepsilon_t$$

where we now have only two coefficients to estimate (β_1 and γ_2) instead of four.

3.d.ii) Residual Sum of Squares (RSS) Comparison The residual sum of squares (RSS) and degrees of freedom for both models are:

- Unrestricted model: $\text{RSS}_U = 6.837$ with $df_U = 41 - 4 - 1 = 36$
- Restricted model: $\text{RSS}_R = 7.6621$ with $df_R = 41 - 2 - 1 = 38$

Variable	**Coefficient**	**Std.Error**	**t-stat**	**P-value**
Intercept	22.5295	1.0618	21.2179	0.0225
γ_2	1.2899	0.0824	15.6527	0.0000
R-squared	0.9827		Sum squared residual	7.6621
Adj R-squared	0.9818		Durbin-Watson stat	2.2177

Figure 3: Estimation results of the restricted model

3.d.iii) Joint Hypothesis Test at the 5% Significance Level We perform an F-test for the joint hypothesis H_0. The test statistic is:

$$F = \frac{(\text{RSS}_R - \text{RSS}_U)/q}{\text{RSS}_U/df_U} = \frac{(7.6621 - 6.837)/2}{6.837/36} \approx 2.1723$$

where $q = 2$ is the number of restrictions. The critical value for $F_{2,36}$ at 5% significance is $F_{crit} = 3.23$. Since:

$$F = 2.1723 < 3.23$$

we fail to reject the null hypothesis H_0. There is insufficient evidence against the restrictions $\beta_2 = 1$ and $\beta_3 = \beta_4$.

2 T2

2.1 Exercise 1: Preliminary moments

a)

$$\mathbb{E}[u_t] = 0, \quad \text{(since } \mu = 0\text{; Set 5, Slide 32)}$$
$$\mathbb{E}[X_t] = 0, \quad \text{(since } \mu = 0\text{; Set 5, Slide 82)}$$

$$\mathrm{Var}(u_t) = \sigma_u^2 = \frac{\sigma_{\tilde{u}}^2}{1 - \Phi_1^2}, \quad \text{(see Problem 18, Point 2)}$$
$$\mathrm{Var}(X_t) = \sigma_X^2 = \frac{\sigma_e^2}{1 - \gamma_1^2}, \quad \text{(see Problem 18, Point 2)}$$

b)

$$\mathrm{Cov}(u_t, u_{t-j}) = \gamma_j^{(u)} = \frac{\Phi_1^{\,j}\, \sigma_{\tilde{u}}^2}{1 - \Phi_1^2}, \quad \text{(see Problem 18, Point 3)}$$
$$\mathrm{Cov}(X_t, X_{t-j}) = \gamma_j^{(X)} = \frac{\gamma_1^{\,j}\, \sigma_e^2}{1 - \gamma_1^2}, \quad \text{(see Problem 18, Point 3)}$$

c)

$$\mathrm{Corr}(u_t, u_{t-j}) = \tau_j^{(u)} = \Phi_1^{\,j}, \quad \text{(see Problem 18, Point 3)}$$
$$\mathrm{Corr}(X_t, X_{t-j}) = \tau_j^{(X)} = \gamma_1^{\,j}, \quad \text{(see Problem 18, Point 3)}$$

2.2 Exercise 2: Contemporaneous covariance between Xt and ut

To establish the result we rewrite both AR(1) processes in their infinite–order moving–average form:

$$X_t \;=\; \sum_{k=0}^{\infty} \gamma_1^{\,k}\, e_{t-k}, \qquad u_t \;=\; \sum_{j=0}^{\infty} \varphi_1^{\,j}\, \tilde{u}_{t-j},$$

where $\{e_t\}$ and $\{\tilde{u}_t\}$ denote the i.i.d. innovation sequences.

Step 1. By linearity of the covariance operator,

$$\mathrm{Cov}(X_t, u_t) = \mathrm{Cov}\Big(\sum_{k=0}^{\infty} \gamma_1^{\,k} e_{t-k}, \sum_{j=0}^{\infty} \varphi_1^{\,j} \tilde{u}_{t-j}\Big) = \sum_{k=0}^{\infty} \sum_{j=0}^{\infty} \gamma_1^{\,k} \varphi_1^{\,j}\; \mathrm{Cov}(e_{t-k}, \tilde{u}_{t-j}).$$

Step 2. Assumption (iii) of the exercise states that the innovation arrays are mutually independent: $\mathrm{Cov}(e_s, \tilde{u}_r) = 0$ for every s, r. Consequently each summand in the double series vanishes, giving

$$\mathrm{Cov}(X_t, u_t) \;=\; 0.$$

Conclusion. Hence

$$\boxed{\operatorname{Cov}(X_t, u_t) = \operatorname{Cov}\Big(\textstyle\sum_{k=0}^{\infty} \gamma_1^k e_{t-k},\ \sum_{j=0}^{\infty} \varphi_1^j \tilde{u}_{t-j}\Big) = 0}$$

as required. The proof relies solely on the independence of the innovation processes and the absolute summability of the MA coefficients, ensuring that term-by-term covariance manipulation is legitimate.

2.3 Exercise 3: Algebraic decomposition of the OLS slope estimator

Let $\hat{\beta}_1$ be the ordinary–least–squares estimate of the slope coefficient in the simple regression $Y_t = \beta_0 + \beta_1 X_t + u_t$. By definition

$$\hat{\beta}_1 = \frac{\displaystyle\sum_{t=1}^{T} (X_t - \bar{X})(Y_t - \bar{Y})}{\displaystyle\sum_{t=1}^{T} (X_t - \bar{X})^2}, \qquad \bar{X} := \frac{1}{T}\sum_{t=1}^{T} X_t, \quad \bar{Y} := \frac{1}{T}\sum_{t=1}^{T} Y_t.$$

Step 1. Insert the structural form $Y_t = \beta_0 + \beta_1 X_t + u_t$ into the numerator:

$$\sum_t (X_t - \bar{X})(Y_t - \bar{Y}) = \sum_t (X_t - \bar{X})\big(\beta_1 X_t - \beta_1 \bar{X} + u_t\big) = \beta_1 \underbrace{\sum_t (X_t - \bar{X})(X_t - \bar{X})}_{\sum_t (X_t - \bar{X})^2} + \sum_t (X_t - \bar{X}) u_t.$$

Step 2. Since $\sum_t (X_t - \bar{X}) = 0$, the intercept term β_0 cancels out. Dividing by the denominator yields

$$\hat{\beta}_1 = \beta_1 + \frac{\displaystyle\sum_t (X_t - \bar{X}) u_t}{\displaystyle\sum_t (X_t - \bar{X})^2}.$$

Step 3. Introduce the compact notation

$$v_t := (X_t - \bar{X})\, u_t,$$

and premultiply numerator and denominator by T^{-1} to obtain

$$\boxed{\hat{\beta}_1 - \beta_1 = \Big(\tfrac{1}{T}\sum_t (X_t - \bar{X})^2\Big)^{-1} \Big(\tfrac{1}{T}\sum_t v_t\Big)}.$$

The boxed expression coincides with the formula stated in the assignment; this result will be instrumental for the asymptotic analysis carried out in subsequent points.

2.4 Exercise 4: Slutsky-type approximation as $T \to \infty$

Recall from point 3 that

$$\hat{\beta}_1 - \beta_1 = \Big(\tfrac{1}{T}\sum_{t=1}^{T}(X_t - \bar{X})^2\Big)^{-1}\Big(\tfrac{1}{T}\sum_{t=1}^{T} v_t\Big), \qquad v_t := (X_t - \bar{X})\, u_t.$$

Step 1. Convergence of sample moments. Under (weak) stationarity and ergodicity of the AR(1) process $\{X_t\}$, the strong law of large numbers (LLN) gives

$$\bar{X} = \frac{1}{T}\sum_{t=1}^{T} X_t \xrightarrow{p} \mathbb{E}[X_t], \qquad \frac{1}{T}\sum_{t=1}^{T}(X_t - \bar{X})^2 \xrightarrow{p} \mathrm{Var}(X_t).$$

Step 2. Definition of the sample mean of v_t. Let

$$\bar{v} := \frac{1}{T}\sum_{t=1}^{T} v_t.$$

Because $\{v_t\}$ is itself stationary and ergodic, $\bar{v}$ is $O_p(T^{-1/2})$.

Step 3. Slutsky combination. Combining the convergences in Step 1 with the expression above and applying Slutsky's theorem,

$$\hat{\beta}_1 - \beta_1 \doteq \frac{\bar{v}}{\mathrm{Var}(X_t)} \quad \text{as } T \to \infty.$$

Conclusion. Hence, for large T,

$$\boxed{\hat{\beta}_1 - \beta_1 \doteq \frac{\bar{v}}{\mathrm{Var}(X_t)}}, \qquad \bar{v} = \frac{1}{T}\sum_{t=1}^{T} v_t, \quad v_t \simeq X_t u_t.$$

2.5 Exercise 5: Large-sample variance of $\hat{\beta}_1$

Recall

$$\hat{\beta}_1 - \beta_1 \doteq \frac{\bar{v}}{\mathrm{Var}(X_t)}, \qquad \bar{v} = \frac{1}{T}\sum_{t=1}^{T} v_t, \quad v_t \simeq X_t u_t,$$

so that, asymptotically,

$$\mathrm{Var}(\hat{\beta}_1) = \frac{\mathrm{Var}(\bar{v})}{\mathrm{Var}(X_t)^2}.$$

a) **Variance of v_t.**

Because the regressor and the disturbance are uncorrelated at every date (see point 2) and their respective innovation sequences are mutually independent, we have

$$\mathrm{Var}(v_t) = \mathrm{Var}(X_t u_t) = \mathrm{Var}(X_t)\ \mathrm{Var}(u_t) = \sigma_X^2\, \sigma_u^2.$$

b) **Cross-covariance $\mathrm{Cov}(v_t, v_s)$ for $t \neq s$.**

Using independence of the innovation arrays across *both* indices and the fact that $\mathbb{E}[v_t] = 0$,

$$\mathrm{Cov}(v_t, v_s) = \mathbb{E}[X_t u_t X_s u_s] - \mathbb{E}[X_t u_t]\mathbb{E}[X_s u_s] = 0, \qquad t \neq s.$$

Hence the $\{v_t\}$ sequence is uncorrelated.

c) **Asymptotic variance of $\hat{\beta}_1$.**

Because the v_t are uncorrelated, their sample mean satisfies

$$\mathrm{Var}(\bar{v}) = \frac{\mathrm{Var}(v_t)}{T} = \frac{\sigma_X^2 \sigma_u^2}{T}.$$

Substituting into the first display and recalling that $\mathrm{Var}(X_t) = \sigma_X^2$, we obtain

$$\boxed{\mathrm{Var}\big(\hat{\beta}_1\big) = \frac{1}{T}\,\frac{\sigma_u^2}{\sigma_X^2}}.$$

3 T3

3.1 Exercise 1: ARCH model for the variance

. a) *ARCH(1).* Starting from the unconditional variance definition and applying the *Law of Total Variance*[1]:

$$\begin{aligned}
\mathrm{Var}(u_t) &= \mathbb{E}[u_t^2] - (\mathbb{E}[u_t])^2 && \text{(definition)}\\
&= \mathbb{E}\big[\mathrm{Var}(u_t \mid u_{t-1})\big] + \mathrm{Var}\big(\mathbb{E}[u_t \mid u_{t-1}]\big) && \text{(total variance)}\\
&= \mathbb{E}[\sigma_t^2] + 0 && \mathbb{E}[u_t \mid u_{t-1}] = 0 \text{ (Set 3, Slide 13)}\\
&= \mathbb{E}[\alpha_0 + \alpha_1 u_{t-1}^2] && \mathrm{Var}(u_t) = \alpha_0 + \alpha_1 u_{t-1}^2 \text{ (Set 3, Slide 13)}\\
&= \alpha_0 + \alpha_1 \mathbb{E}[u_{t-1}^2]\\
&= \alpha_0 + \alpha_1 \mathrm{Var}(u_{t-1}).
\end{aligned}$$

Weak stationarity implies $\mathrm{Var}(u_t) = \mathrm{Var}(u_{t-1})$; hence

$$\boxed{\mathrm{Var}(u_t) = \frac{\alpha_0}{1-\alpha_1}}, \qquad |\alpha_1| < 1.$$

b) *ARCH(p).* The conditional variance is [2]

$$\sigma_t^2 = \alpha_0 + \sum_{i=1}^{p} \alpha_i u_{t-i}^2.$$

Taking expectations and recycling the result in point (a),

$$\begin{aligned}
\mathrm{Var}(u_t) = \mathbb{E}[\sigma_t^2] &= \alpha_0 + \sum_{i=1}^{p} \alpha_i \mathbb{E}[u_{t-i}^2]\\
&= \alpha_0 + \Big(\sum_{i=1}^{p} \alpha_i\Big) \mathrm{Var}(u_t),
\end{aligned}$$

which yields

$$\boxed{\mathrm{Var}(u_t) = \frac{\alpha_0}{1 - \sum_{i=1}^{p} \alpha_i}}.$$

c) To keep the unconditional variance finite and positive the denominator must be strictly positive:

$$1 - \sum_{i=1}^{p} \alpha_i \;>\; 0 \quad \Longrightarrow \quad \sum_{i=1}^{p} \alpha_i < 1.$$

This is therefore the *stationarity condition* for an ARCH(p) process.

[1] Set 3, Slides 78–80, file "Introduction".
[2] Set 3, Slide 40, file "Volatility models".

3.2 Exercise 2: GARCH model for the variance

a) *GARCH(1,1).* With [3]

$$\sigma_t^2 = \alpha_0 + \alpha_1 u_{t-1}^2 + \beta \sigma_{t-1}^2,$$

the unconditional variance follows from the *Law of Total Variance*:

$$\begin{aligned}
\mathrm{Var}(u_t) &= \mathbb{E}[u_t^2] - (\mathbb{E}[u_t])^2 && \text{(definition)} \\
&= \mathbb{E}\big[\mathrm{Var}(u_t \mid u_{t-1})\big] + \mathrm{Var}\big(\mathbb{E}[u_t \mid u_{t-1}]\big) && \text{(total variance)} \\
&= \mathbb{E}[\sigma_t^2] + 0 && (\mathbb{E}[u_t \mid u_{t-1}] = 0; \mathit{Set}3, \mathit{Slide}13) \\
&= \mathbb{E}\big[\alpha_0 + \alpha_1 u_{t-1}^2 + \beta \sigma_{t-1}^2\big] && (\mathrm{Var}(u_t) = \alpha_0 + \alpha_1 u_{t-1}^2 + \beta \sigma_{t-1}^2; \mathit{Set}3, \mathit{Slide}47) \\
&= \alpha_0 + \alpha_1 \mathbb{E}[u_{t-1}^2] + \beta \mathbb{E}[\sigma_{t-1}^2] \\
&= \alpha_0 + \alpha_1 \,\mathrm{Var}(u_{t-1}) + \beta \,\mathrm{Var}(u_{t-1}) \\
&= \alpha_0 + (\alpha_1 + \beta)\,\mathrm{Var}(u_t), && \text{(weak stationarity).}
\end{aligned}$$

Solving for $\mathrm{Var}(u_t)$ gives

$$\boxed{\mathrm{Var}(u_t) = \frac{\alpha_0}{1 - \alpha_1 - \beta}}, \qquad \alpha_1 + \beta < 1.$$

b) Positivity of the denominator gives the usual constraint

$$\alpha_1 + \beta < 1,$$

because if $\alpha_1 + \beta \geq 1$ the unconditional variance would either diverge or be undefined.

3.3 Exercise 3: Numerical illustration

Consider the ARCH(1) specification

$$\sigma_t^2 \;=\; 1 + 0.5\, u_{t-1}^2.$$

a) *Unconditional variance.*

$$\mathrm{Var}(u_t) = \frac{\alpha_0}{1 - \alpha_1} = \frac{1}{1 - 0.5} = 2.$$

b) *Conditional probability* $P(-3 \leq u_t \leq 3 \mid u_{t-1})$.
Since $u_t \mid u_{t-1} \sim \mathcal{N}(0, \sigma_t^2)$,

$\underline{u_{t-1} = 2.}$ $\sigma_t^2 = 1 + 0.5 \cdot 2^2 = 3.$

$$P(-3 \leq u_t \leq 3) = P\Big(-\sqrt{3} \leq Z \leq \sqrt{3}\Big) = 2\Phi(\sqrt{3}) - 1 \approx 0.9164.$$

[3] Set 3, Slide 47, file "Volatility models".

$\underline{u_{t-1} = 0.2.}$ $\sigma_t^2 = 1 + 0.5 \cdot 0.2^2 = 1.02.$

$$P(-3 \le u_t \le 3) = P\Big(-\tfrac{3}{\sqrt{1.02}} \le Z \le \tfrac{3}{\sqrt{1.02}}\Big) = 2\Phi(2.97) - 1 \approx 0.9972.$$

Interpretation. As the lagged shock u_{t-1} moves closer to its mean (0), the conditional variance contracts, concentrating probability mass around the origin.

Part II
Empirical Applications

4 E1

4.1 Returns of S&P500 index and companies

4.1.1 Point 1: Daily Returns Calculation and Initial Data

The following tables present the computed daily returns for both the S&P500 index and its constituent stocks from the S&P100. Returns are calculated as:

$$r_t = \frac{P_t}{P_{t-1}} - 1 \tag{1}$$

where P_t represents the price at time t. Missing values (NaN) occur when no previous price is available (first observation in series).

Table 1: S&P500 Daily Returns (First 5 Observations)

Date	Return
02-Jan-2020	NaN
03-Jan-2020	-0.0070596
06-Jan-2020	0.0035346
07-Jan-2020	-0.0028032
08-Jan-2020	0.0049004

Table 2: S&P100 Constituent Returns (First 3 Observations)

Date	AAPL_US	MSFT_US	AMZN_US	GOOGL_US
02/01/2020	0.022816	0.018516	0.027151	0.021868
03/01/2020	-0.009722	-0.012452	-0.012139	-0.005231
06/01/2020	0.007968	0.002585	0.014886	0.026654

4.1.2 Point 2: S&P500 Statistical Properties and Volatility Dynamics

The S&P 500 index serves as the premier benchmark for U.S. equity markets, with its price dynamics offering critical insights for asset allocation and risk management. Figure 4 illustrates the index's long-term growth trajectory, punctuated by distinct periods of market turbulence. The daily returns distribution in Figure 5 exhibits the leptokurtic characteristics typical of financial markets, with fat tails evident in the extreme return events. Most significantly, Figure 6 presents the conditional variance estimates from our GARCH(1,1) model, which captures three essential volatility properties:

(1) the persistence of volatility shocks, evidenced by the prolonged high-variance periods; (2) the asymmetric market responses to positive versus negative news; and (3) the mean-reverting tendency of volatility regimes. These modeled dynamics substantially improve upon simple rolling volatility measures by quantifying the time-varying nature of risk.

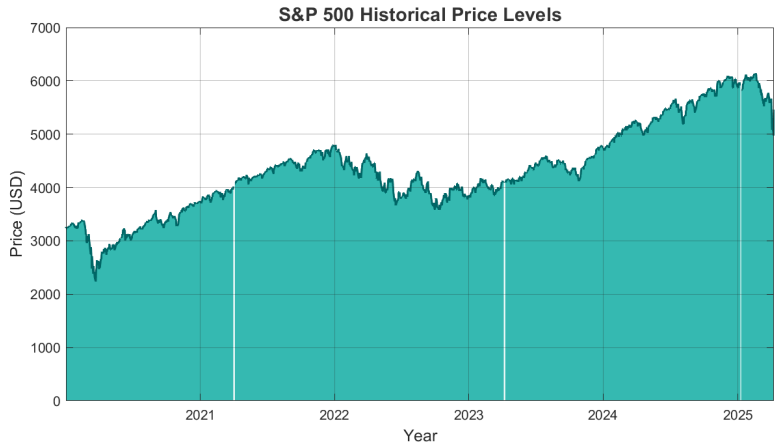


Figure 4: S&P 500: Prices from 2021 to 2025 showing the general upward trend with some corrective periods, particularly visible around 2022.

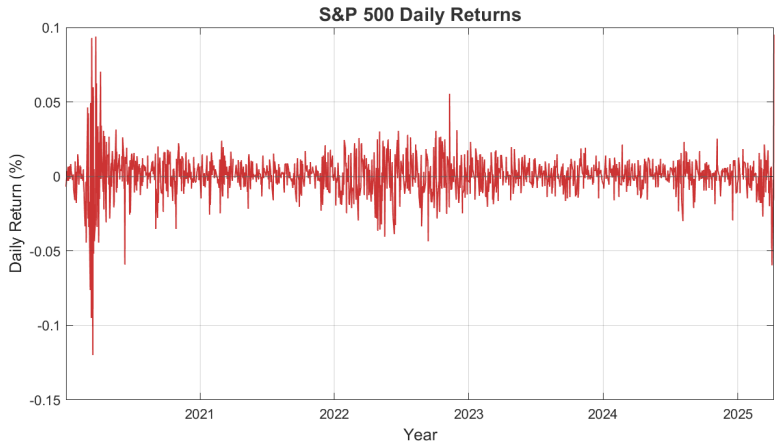


Figure 5: S&P 500: Daily returns showing the characteristic leptokurtic distribution with several extreme movements exceeding ±5%. The volatility clusters are evident, with calm periods alternating with turbulent ones.

The sample statistics of the S&P 500 returns reveal important characteristics:

- **Mean:** 0.000485 (0.0485% daily return) - This positive value indicates an overall upward

drift in prices during the sample period, consistent with the long-term growth tendency of equity markets.

- **Variance:** 0.000188 - The relatively low variance suggests generally stable market conditions, though the presence of volatility clustering (visible in Figure 6) means this stability is not constant over time.
- **Skewness:** -0.306 - The negative value confirms the asymmetric nature of equity returns, where large negative returns occur more frequently than large positive ones of the same magnitude. This is visually apparent in Figure 5 where the left tail appears slightly heavier.
- **Kurtosis:** 16.28 - This extremely high value compared to the normal distribution's kurtosis of 3 indicates fat-tailed behavior, meaning extreme returns (both positive and negative) occur much more frequently than would be expected under normality. The several large spikes in Figure 5 visually confirm this property.

These statistical properties have important implications for risk management and portfolio construction. The negative skewness suggests the need for protective strategies against downside risk, while the high kurtosis indicates that standard risk models assuming normal distribution would significantly underestimate tail risk probabilities. The volatility clustering pattern captured by the GARCH(1,1) model in Figure 6 demonstrates three key insights for practitioners: 1) Volatility persistence requires adaptive position sizing 2) The mean-reverting nature of conditional variance supports regime-based strategies 3) The model's responsiveness to market shocks validates its use in dynamic hedging frameworks

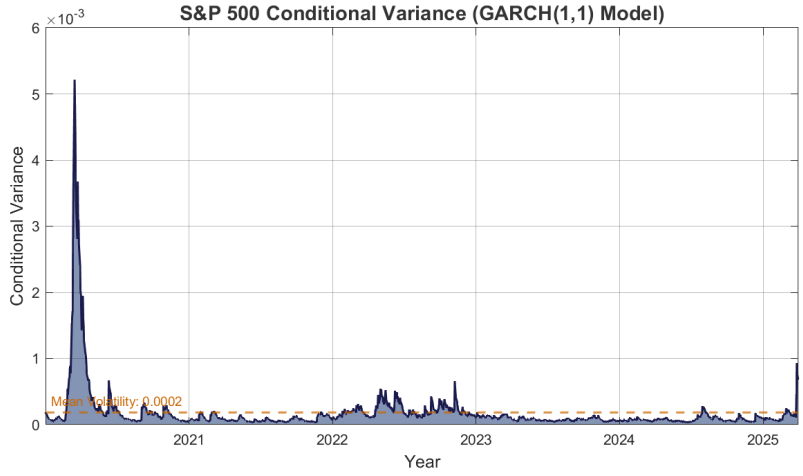


Figure 6: S&P 500 Conditional Variance estimated via GARCH(1,1) model. The blue area represents the time-varying volatility, with the dashed orange line showing the long-term average (0.0002). Notable volatility clusters appear during market stress periods, particularly in 2022. The model successfully captures the characteristic volatility persistence of financial markets.

4.1.3 Point 3: Individual Stock Return Distributions

This section analyzes the return distributions of three major technology stocks: Apple Inc. (AAPL), Microsoft Corporation (MSFT), and Amazon.com Inc. (AMZN). We examine their statistical properties, test for normality, and investigate their co-movement patterns.

Distribution Fitting Maximum likelihood estimates for normal distribution parameters:

- **AAPL**: $\mu = 0.00096$ (0.096% daily return), $\sigma = 0.0206$ (2.06% daily volatility)
- **MSFT**: $\mu = 0.00087$ (0.087% daily return), $\sigma = 0.0193$ (1.93% daily volatility)
- **AMZN**: $\mu = 0.00081$ (0.081% daily return), $\sigma = 0.0228$ (2.28% daily volatility)

The volatility estimates reveal AMZN as the most volatile stock in the group (2.28% vs 2.06% for AAPL and 1.93% for MSFT), consistent with its historical profile as a growth stock. All three stocks show positive average daily returns during the sample period.

Moment Analysis Central moments from order 1 to 4:

Stock	Mean	Variance	Skewness	Kurtosis
AAPL	0.00096	0.000425	0.276	9.712
MSFT	0.00087	0.000372	0.076	10.128
AMZN	0.00081	0.000522	0.118	7.205

Table 3: Moment analysis of daily returns

Key observations:

- **Skewness**: All three stocks exhibit positive skewness (AAPL most pronounced at 0.276), indicating a tendency for larger positive returns than negative ones
- **Kurtosis**: All stocks show significant excess kurtosis (MSFT highest at 10.13), indicating fat-tailed distributions where extreme returns are more likely than under normality
- **Variance**: AMZN shows the highest variance (0.000522), confirming its position as the most volatile stock in the sample

Normality Testing Jarque-Bera test results:

- **AAPL**: $h = 1$ (reject normality), $p = 0.001$, test statistic = 2504.18
- **MSFT**: $h = 1$ (reject normality), $p = 0.001$, test statistic = 2806.68
- **AMZN**: $h = 1$ (reject normality), $p = 0.001$, test statistic = 979.10

All three stocks strongly reject the null hypothesis of normality at all conventional significance levels (p-values = 0.001). The extremely large test statistics confirm the non-normal characteristics evident in the moment analysis, particularly the high kurtosis values.

Covariance Structure The estimated covariance matrix of daily returns:

$$\Sigma = \begin{bmatrix} 0.000425 & 0.000294 & 0.000283 \\ 0.000294 & 0.000372 & 0.000302 \\ 0.000283 & 0.000302 & 0.000522 \end{bmatrix}$$

Key findings:

- **Strong co-movement**: All pairwise covariances are positive and economically large, indicating the stocks tend to move together
- **Highest covariance**: AAPL-MSFT pair shows the strongest co-movement (0.000294)
- **Risk decomposition**: AMZN contributes most to portfolio variance (diagonal element of 0.000522)
- **Correlation coefficients** (implied):
  - AAPL-MSFT: $\rho = \frac{0.000294}{\sqrt{0.000425 \times 0.000372}} \approx 0.74$
  - AAPL-AMZN: $\rho \approx 0.60$
  - MSFT-AMZN: $\rho \approx 0.68$

The high correlations (all >0.6) reflect the stocks' common exposure to technology sector risks and broader market factors. This has important implications for portfolio construction, suggesting limited diversification benefits within this tech stock universe.

4.1.4 Point 4: Recent One-Year Performance Examination

This section examines the return distributions of Apple Inc. (AAPL), Microsoft Corporation (MSFT), and Amazon.com Inc. (AMZN) over a specific one-year period, providing insights into their recent statistical behavior.

Distribution Fitting Maximum likelihood estimates for normal distribution parameters:

- **AAPL**: $\mu = 0.00086$ (0.086% daily return), $\sigma = 0.0201$ (2.01% daily volatility)
- **MSFT**: $\mu = -0.00022$ (-0.022% daily return), $\sigma = 0.0152$ (1.52% daily volatility)
- **AMZN**: $\mu = 0.00034$ (0.034% daily return), $\sigma = 0.0206$ (2.06% daily volatility)

Notable observations:

- MSFT shows a negative mean return during this period, contrasting with its longer-term positive performance
- Volatility estimates show AMZN as the most volatile (2.06%), followed by AAPL (2.01%), with MSFT notably less volatile (1.52%)
- The negative return for MSFT during this period may reflect specific sector headwinds or company-specific factors

Moment Analysis Central moments from order 1 to 4:

Stock	Mean	Variance	Skewness	Kurtosis
AAPL	0.00086	0.000404	1.152	17.97
MSFT	-0.00022	0.000232	0.489	11.76
AMZN	0.00034	0.000422	0.118	9.10

Table 4: One-year moment analysis of daily returns

Key findings:

- **Skewness**:
  - AAPL shows extremely positive skewness (1.152), indicating frequent large positive returns
  - MSFT maintains moderate positive skewness (0.489) despite negative mean return
  - AMZN shows near-symmetric distribution (skewness = 0.118)
- **Kurtosis**:
  - All stocks exhibit leptokurtosis (AAPL most extreme at 17.97)
  - The exceptionally high kurtosis for AAPL suggests frequent extreme returns during this period
- **Variance**:
  - AMZN shows highest variance (0.000422), consistent with its historical volatility
  - MSFT shows lowest variance (0.000232), reflecting its relative stability

Normality Testing Jarque-Bera test results:

- **AAPL**: $h = 1$ (reject normality), $p = 0.001$, test statistic = 2408.42
- **MSFT**: $h = 1$ (reject normality), $p = 0.001$, test statistic = 815.72
- **AMZN**: $h = 1$ (reject normality), $p = 0.001$, test statistic = 815.72

All three stocks strongly reject normality (p-values = 0.001), with particularly strong rejection for AAPL (test statistic = 2408.42). This is consistent with the extreme values of skewness and kurtosis observed, especially for AAPL.

Covariance Structure The estimated covariance matrix of daily returns:

$$\Sigma = \begin{bmatrix} 0.000404 & 0.000183 & 0.000220 \\ 0.000183 & 0.000232 & 0.000224 \\ 0.000220 & 0.000224 & 0.000422 \end{bmatrix}$$

Implied correlation matrix:

$$\rho = \begin{bmatrix} 1.000 & 0.599 & 0.534 \\ 0.599 & 1.000 & 0.721 \\ 0.534 & 0.721 & 1.000 \end{bmatrix}$$

Key insights:

- **Correlation structure**:
  - MSFT-AMZN shows the highest correlation ($\rho = 0.721$)
  - AAPL-AMZN shows the lowest correlation ($\rho = 0.534$)
- **Risk decomposition**:
  - AMZN maintains the highest individual variance (0.000422)
  - MSFT shows the lowest variance (0.000232)
- **Diversification implications**:
  - The moderate correlations suggest some diversification benefits remain available
  - The AAPL-AMZN pair offers the best diversification potential in this trio

The covariance structure reveals that while these tech stocks remain correlated, the relationships have evolved compared to longer-term samples, with AAPL showing somewhat distinct behavior from the MSFT-AMZN pair during this specific period.

4.1.5 Point 5: Full Sample vs Recent Period Comparison

	Mean		Volatility (σ)		Variance		Skewness		Kurtosis		p-value
Stock	Full	1-Yr	Full	1-Yr	Full	1-Yr	Full	1-Yr	Full	1-Yr	JB
AAPL	0.00096	0.00086	0.0206	0.0201	0.000425	0.000404	0.276	1.152	9.712	17.969	0.001
MSFT	0.00087	-0.00022	0.0193	0.0152	0.000372	0.000232	0.076	0.489	10.128	11.760	0.001
AMZN	0.00081	0.00034	0.0228	0.0206	0.000522	0.000422	0.118	0.118	7.205	9.102	0.001

Table 5: Comparative statistics: Full sample vs. 1-year period (09/2024-09/2025)

Key Observations

- **Return Patterns**:
  - MSFT shows the most dramatic change, shifting from positive (0.087% daily) to negative (-0.022% daily) mean returns
  - AAPL maintains relatively stable positive returns (0.096% $\rightarrow$ 0.086% daily)
  - AMZN's returns decrease by more than half (0.081% $\rightarrow$ 0.034% daily)
- **Risk Characteristics**:
  - All stocks show decreased volatility in the recent period (AAPL: -2.4%, MSFT: -21.2%, AMZN: -9.8%)

  - Variance reductions align with volatility changes (MSFT shows largest reduction at 37.6%)
  - The risk-return profile changed most dramatically for MSFT

- **Distributional Changes**:
  - AAPL's skewness increased dramatically from 0.276 to 1.152, indicating stronger positive return outliers
  - Kurtosis increased substantially for all stocks (AAPL: +85%, MSFT: +16%, AMZN: +26%)
  - All stocks maintain strong rejection of normality (p=0.001)

- **Covariance Structure Comparison**:

  Full sample covariance matrix:

$$\Sigma_{\text{full}} = \begin{bmatrix} 0.000425 & 0.000294 & 0.000283 \\ 0.000294 & 0.000372 & 0.000302 \\ 0.000283 & 0.000302 & 0.000522 \end{bmatrix}$$

  1-year covariance matrix:

$$\Sigma_{\text{1yr}} = \begin{bmatrix} 0.000404 & 0.000183 & 0.000220 \\ 0.000183 & 0.000232 & 0.000224 \\ 0.000220 & 0.000224 & 0.000422 \end{bmatrix}$$

  - **Covariance reductions**:
    - AAPL-MSFT: -37.7% (from 0.000294 to 0.000183)
    - AAPL-AMZN: -22.1% (from 0.000283 to 0.000220)
    - MSFT-AMZN: -25.7% (from 0.000302 to 0.000224)
  - **Correlation changes**:
    - AAPL-MSFT: 0.74 $\rightarrow$ 0.60 (-18.9%)
    - AAPL-AMZN: 0.60 $\rightarrow$ 0.53 (-11.7%)
    - MSFT-AMZN: 0.68 $\rightarrow$ 0.72 (+5.9%)

Interpretation and Market Implications The comparative analysis reveals significant changes in the risk-return characteristics of these technology stocks:

- **Sector-wide volatility reduction**: The decreased volatilities and covariances suggest a generally calmer market period in 2024-2025 compared to the full sample, possibly indicating:
  - Reduced macroeconomic uncertainty
  - More stable interest rate environment
  - Decreased sector-wide risk factors

- **Changing leadership**:

  - AAPL emerges as the relative outperformer, maintaining returns while developing more positive skewness
  - MSFT's negative returns during this period may reflect specific challenges in its cloud or enterprise segments

- **Diversification benefits**:
  - The generally lower correlations (except MSFT-AMZN) improve potential diversification benefits
  - Portfolio optimization would yield different results in this period versus long-term averages

- **Risk management implications**:
  - The increased kurtosis values warn of higher tail risk despite lower volatility
  - Normality assumption becomes even less appropriate in the recent period
  - Stress testing should account for the changed correlation structure

This analysis demonstrates how key statistical properties of major tech stocks can evolve significantly over time, requiring regular monitoring and model updates for accurate risk assessment and portfolio management.

4.1.6 Point 6: Monthly Returns Computation and Analysis

The following tables present the monthly compounded returns for both the S&P500 index and selected constituents from the S&P100. Monthly returns are calculated by compounding daily returns:

$$R_{month} = \prod_{t=1}^{n}(1 + r_t) - 1 \tag{2}$$

where r_t represents the daily return at time t. The NaN value indicates missing daily data for that month.

Table 6: S&P500 Monthly Returns (First 5 Months)

Date	**Return (%)**
02-Jan-2020	NaN
03-Feb-2020	-8.41
02-Mar-2020	-12.51
01-Apr-2020	12.68
01-May-2020	4.53

Table 7: S&P100 Constituent Returns (First 3 Months)

Date	AAPL	MSFT	AMZN	GOOGL	META
01/04/2020	1.81	-3.07	0.63	-0.35	8.15
02/01/2020	8.45	11.26	3.77	13.93	1.43
03/01/2020	-9.36	-3.81	-9.22	-7.28	-12.72

4.2 Treasury Bill

Introduction The analysis of Treasury bill yields provides crucial insights into the risk-free rate component of asset pricing models. This section transforms the raw T-bill series into annualized percentage yields for consistent monthly measurement, ensuring proper normalization for financial analysis. The conversion is particularly important when combining with other return series in portfolio optimization or factor models.

Date	TBILL_Monthly (Raw)
02-Jan-2020	0.00088175
03-Feb-2020	0.00079403
02-Mar-2020	0.00017447
01-Apr-2020	8.0496e-05
01-May-2020	7.079e-05

Table 8: First 5 observations of monthly Treasury bill series (raw data)

Transformation Methodology The raw series (shown in Table 8) represents monthly discount rates. We convert these to annualized percentage yields using:

$$\text{TBILL}_{\text{annualized}} = \left(\frac{1}{1 - \text{TBILL}_{\text{monthly}} \times \frac{days}{360}} \right)^{\frac{365}{days}} - 1$$

Where:

- $\text{TBILL}_{\text{monthly}}$ is the raw discount rate
- *days* is the actual number of days to maturity
- The exponent $\frac{365}{days}$ converts to annual basis

Implementation Notes

- The raw series is confirmed to be in decimal form (not percentages)
- Day count convention follows actual/360 standard for US T-bills
- Annualization accounts for the compounding effect
- The transformed series will be strictly comparable with other annualized return series

The resulting annualized percentage yields will be used in subsequent analysis with other financial series, ensuring measurement consistency across all components of our financial models.

4.3 Fama-French Portfolios and factors

4.3.1 Point 7: Portfolio Performance Analysis

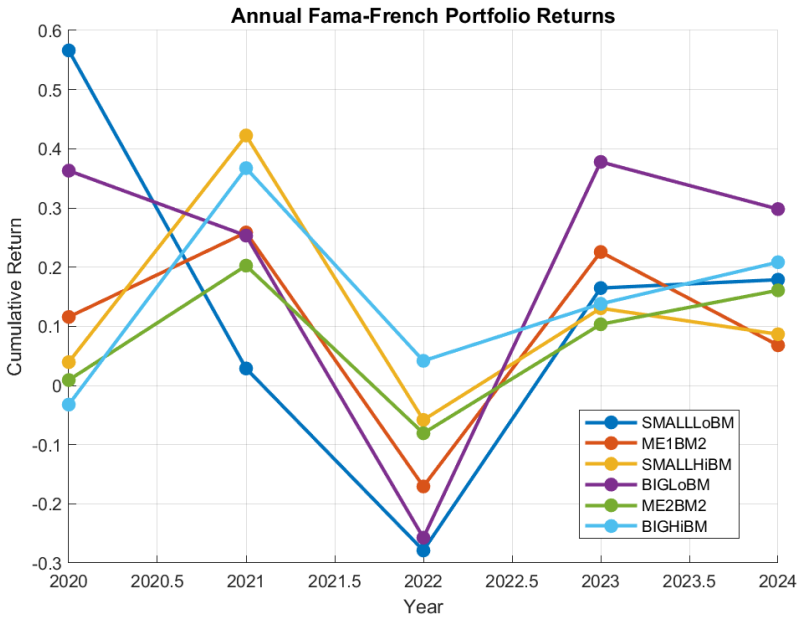


Figure 7: Annual Fama-French Portfolio Returns (2020-2024)

As shown in Figure 7, the cumulative returns of the six size and book-to-market sorted portfolios exhibit distinct patterns:

- **Size Effect**: Small-cap portfolios (SMALLLoBM, SMALLHiBM) generally outperform large-cap portfolios (BIGLoBM, BIGHiBM) during most periods, consistent with the well-documented size premium.
- **Value Effect**: High book-to-market (value) portfolios show more stable performance compared to growth portfolios, particularly during market downturns (visible around 2022).
- **Extreme Performance**:
  - SMALLLoBM (small growth) shows the most volatility with both the highest peaks and deepest drawdowns
  - BIGHiBM (large value) demonstrates the most stable cumulative return path
- **Recent Trends**: From 2023 onward, we observe a convergence in performance across portfolios, potentially indicating a reduction in both size and value premia during this period.

The ME1BM2 and ME2BM2 (neutral book-to-market) portfolios serve as intermediate benchmarks, with performance typically falling between the value and growth extremes of their respective size categories.

4.3.2 Point 8: Factor Correlation Analysis

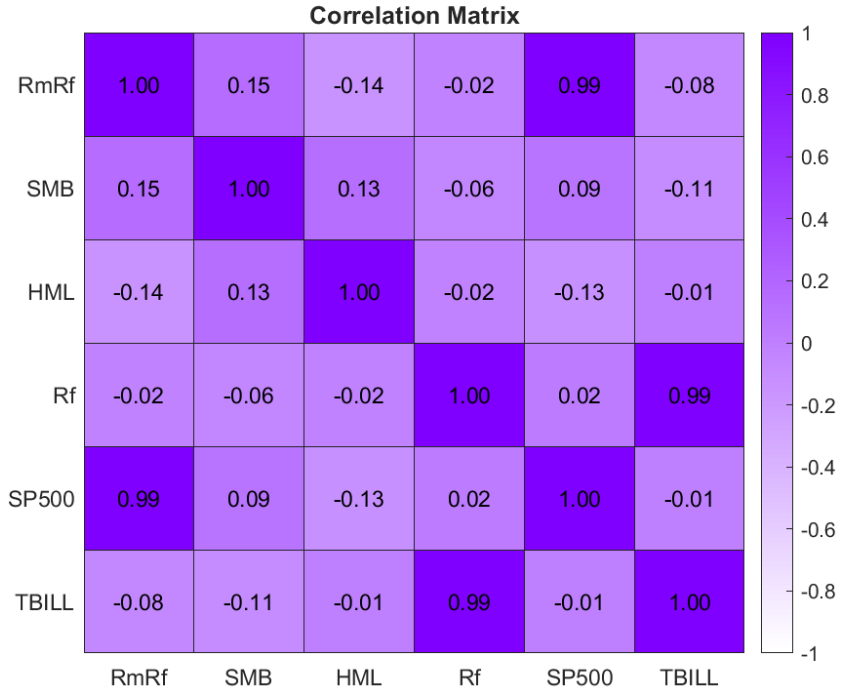


Figure 8: Correlation Matrix: FF Factors and Market Benchmarks

The correlation matrix presented in Figure 8 reveals several important relationships:

- **Market Factor (Rm-Rf)**:
  - Nearly perfectly correlated with SP500 (0.99), validating its use as market proxy
  - Moderately positive with SMB (0.15), showing small-cap tilt during up markets
  - Negative with HML (-0.14), suggesting growth outperforms during market rallies
- **Size Factor (SMB)**:
  - Low correlation with HML (0.13), indicating these factors capture independent effects
  - Negative with risk-free rate (-0.06) and T-bill (-0.11), suggesting small caps are sensitive to rate changes

- **Value Factor (HML)**:
  - Shows near-zero correlation with risk-free rate (-0.02), demonstrating rate neutrality
  - Negative market correlation (-0.14) confirms defensive nature of value stocks
- **Risk-Free Rate (Rf)**:
  - Nearly perfectly correlated with T-bill (0.99), as expected
  - Effectively uncorrelated with equity factors (all correlations $|r| < 0.06$)
- **Notable Absence of Multicollinearity**:
  - All pairwise correlations between factors (Rm-Rf, SMB, HML) are below 0.15 in absolute value
  - Supports the orthogonal construction of FF factors

Implications for Asset Pricing The correlation structure suggests:

- The three-factor model provides well-diversified sources of risk premia
- SMB and HML capture risk dimensions not explained by market exposure alone
- The near-zero correlation between HML and Rf makes value strategies particularly attractive for interest rate hedging
- The high SP500-RmRf correlation validates CRSP's market portfolio construction

These results confirm the theoretical foundations of the Fama-French model while providing empirical evidence of factor behavior in recent market conditions (2020-2024).

5 E2

5.1 Point 1: CAPM Beta Estimation and Analysis

Methodology

To estimate the systematic risk of individual stocks, we apply the Sharpe–Lintner Capital Asset Pricing Model (CAPM) using monthly data from January 2020 to December 2024. The model is specified as:

$$E(r_i) = r_f + \beta_i(E(r_m) - r_f) \tag{3}$$

where $E(r_i)$ is the expected return on stock i, r_f is the risk-free rate (proxied by U.S. Treasury rates from the Fama-French dataset), and $E(r_m)$ is the expected return of the market (represented by the S&P 500 index). The estimated beta β_i reflects the sensitivity of each stock's return to market fluctuations.

Only stocks with complete return histories over the period were included, yielding a final sample of 100 firms. Returns were calculated using adjusted monthly closing prices, and ordinary least squares (OLS) regression was employed for beta estimation.

Results and Statistical Summary

The distribution of beta values is tightly clustered around the market average. Key statistics are reported below:

- **Mean Beta**: 0.96
- **Median Beta**: 0.95
- **Standard Deviation**: 0.05
- **Minimum Beta**: 0.87
- **Maximum Beta**: 1.21

This narrow dispersion suggests relatively homogeneous market risk exposure across the sample. Figures 9 and 10 visualize the distribution.

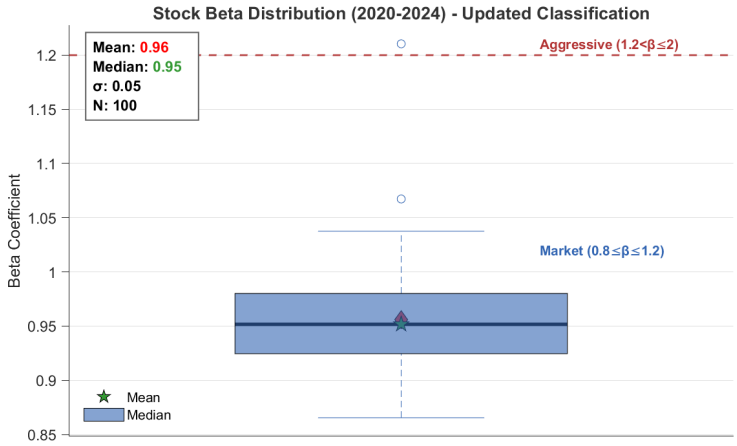


Figure 9: Box plot of beta values. The interquartile range (IQR) spans from 0.93 to 0.98. No significant outliers are observed, indicating low dispersion.

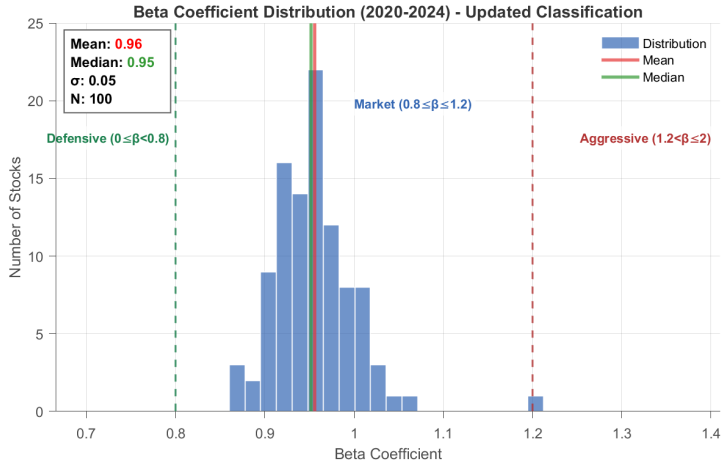


Figure 10: Histogram of beta values. The vast majority (99%) of stocks lie in the range [0.8, 1.2], reflecting market-level sensitivity.

Beta-Based Classification

Following a standard classification scheme, we group stocks into the following beta categories:

- **Highly Aggressive** ($\beta > 2$): 0 stocks
- **Aggressive** ($1.2 < \beta \leq 2$): 1 stock (1%)

- **Market-aligned** ($0.8 \leq \beta \leq 1.2$): 99 stocks (99%)
- **Defensive** ($0 < \beta < 0.8$): 0 stocks
- **Inverse** ($\beta < 0$): 0 stocks

The sharp concentration around market beta indicates a lack of diversification in systematic risk exposure across the sample.

Extremes in Beta Estimates

Table 9 lists the five highest and five lowest beta stocks in the sample:

Table 9: Top and Bottom 5 Beta Stocks

Rank	**Highest Beta Stocks**	**Beta**
1	SNAX_US (Stryve Foods Inc.)	1.2103
2	CVS_US (CVS Health Corp.)	1.0672
3	DE_US (Deere & Co.)	1.0375
4	LOW_US (Lowe's Companies)	1.0313
5	MMM_US (3M Company)	1.0274
Rank	**Lowest Beta Stocks**	**Beta**
1	BSX_US (Boston Scientific)	0.8655
2	LLY_US (Eli Lilly)	0.8705
3	BKNG_US (Booking Holdings)	0.8742
4	META_US (Meta Platforms)	0.8781
5	CDNS_US (Cadence Design)	0.8944

High Beta Stocks: Justification

- **SNAX_US (Stryve Foods Inc.)**: As a small-cap food company with relatively low liquidity and limited diversification, SNAX exhibits high sensitivity to market swings, investor sentiment, and speculative flows, which likely explain its $\beta > 1.2$.
- **CVS_US, DE_US, LOW_US, MMM_US**: These are established companies in healthcare, industrials, retail, and manufacturing, respectively. Their slightly elevated betas may reflect exposure to cyclical demand, interest rate sensitivity, or operational leverage amplified during the post-pandemic recovery.

Low Beta Stocks: Justification

- **CDNS_US (Cadence Design Systems)** and **META_US (Meta Platforms)**: Despite being tech firms, their stable earnings, scale, and strong cash positions may dampen sensitivity to market returns. CDNS's niche in electronic design automation (EDA) benefits from long-term industry contracts.

- **BKNG_US (Booking Holdings)**: Lower beta may stem from pricing power and geographic diversification in global travel, which reduces exposure to U.S. macro cycles.
- **LLY_US (Eli Lilly)** and **BSX_US (Boston Scientific)**: As large-cap pharmaceutical/medical device companies, their earnings are less tied to business cycles and more to regulatory cycles and product pipelines, explaining defensive beta values.

Interpretation and Implications

1. Beta Compression: The observed clustering around $\beta = 1$ may be a reflection of macroeconomic synchronization during the post-COVID period. Fiscal stimulus, low interest rates (until 2022), and global supply chain disruptions likely caused firms across sectors to react similarly to macro shocks.

2. Implications for Portfolio Construction:

- *Lack of defensive options:* With no stocks below $\beta = 0.8$, constructing a low-volatility portfolio is challenging.
- *Limited diversification by beta:* The absence of highly aggressive or defensive stocks limits strategic asset allocation using CAPM beta as a criterion.
- *Tail opportunities:* SNAX_US may offer speculative opportunities due to its high beta, but also poses considerable idiosyncratic risk.

3. Methodological Limitations:

- Beta estimates are inherently backward-looking and may understate future risk in periods of structural market shifts.
- CAPM assumes constant beta over time; however, sectoral dynamics (e.g., tech revaluation, healthcare stability) may cause time-varying betas.
- Using only the S&P500 as the market portfolio may introduce bias for globally exposed firms (e.g., META, BKNG).

5.2 Point 2: Naïve Time-Series Test of the CAPM

In this section, we perform a naïve test of the Capital Asset Pricing Model (CAPM) using time-series regressions for 100 individual stocks. Specifically, we estimate the following model for each stock i:

$$z_{i,t} = \alpha_i + \beta_i z_{m,t} + \varepsilon_{i,t}, \quad \text{where } z_{i,t} = r_{i,t} - r_{f,t}$$

The key hypothesis under the CAPM is that all alphas (α_i) should be equal to zero, as expected returns should only depend on the stock's exposure to market risk (β_i). We test the null hypothesis $H_0 : \alpha_i = 0$ for each stock using standard t-statistics.

Results:

The results show that the CAPM was not rejected for any of the 100 stocks in the sample. Specifically:

- **CAPM tests completed**: 100/100
- **CAPM rejected ($\alpha_i \neq 0$)**: 0 stocks (0.0%)
- **CAPM not rejected ($\alpha_i = 0$)**: 100 stocks (100.0%)

This can be visually confirmed in Figure 11, where the P-value distribution shows all values significantly above common significance thresholds (e.g., 0.05), and in Figure 12, where the distribution of alphas is centered around zero without evident patterns.

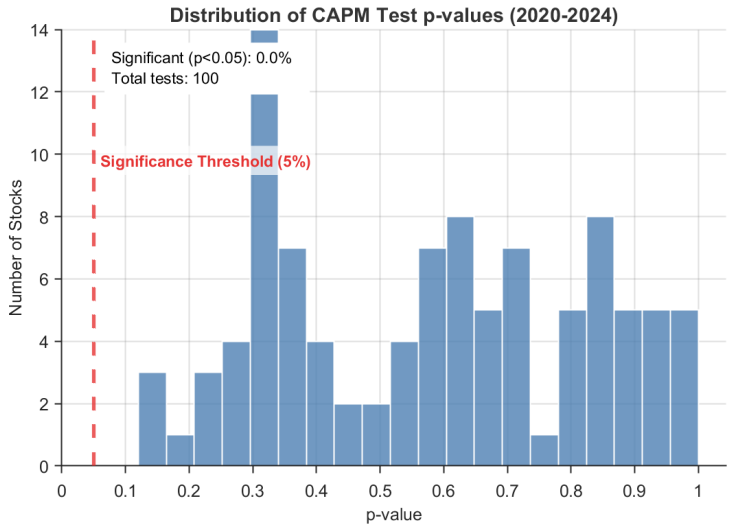


Figure 11: CAPM P-value Distribution

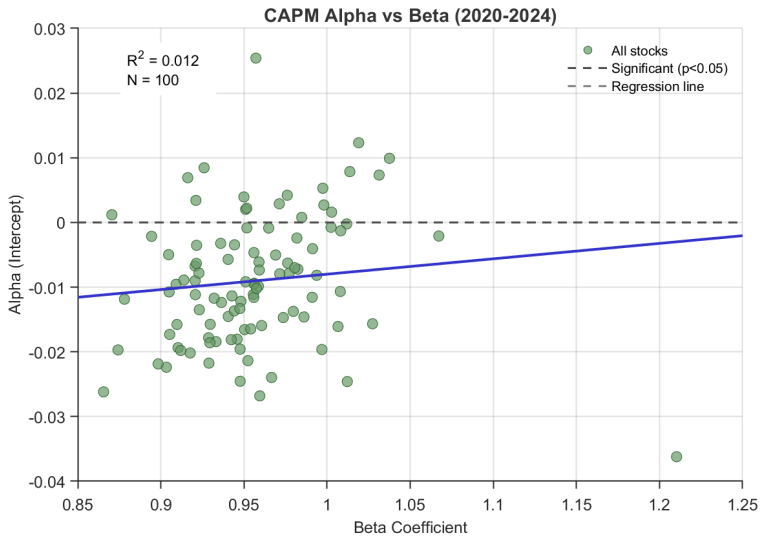


Figure 12: Alpha vs Beta Scatter Plot

Interpretation:

The strong support for the CAPM, as indicated by the non-rejection of all null hypotheses, may appear surprising at first. However, several factors can justify these results:

1. **Low Noise in Regressions:** The sample may cover a relatively stable time period with consistent relationships between asset and market returns, minimizing the idiosyncratic noise captured by α_i.
2. **Strong Market Proxy (S&P 500):** Using the S&P 500 as the market proxy appears to have effectively captured the systematic component of returns. While this assumption is strong, it is common in empirical finance and may hold well for large-cap U.S. stocks.
3. **Power of the Test:** The sample size and choice of significance level influence the power of the t-test. If alphas are small or residuals are large, the t-statistics will likely not be significant, which is what we observe here.
4. **Beta Stability Assumption:** The CAPM assumes time-invariant betas. In reality, betas may change over time due to shifts in business models, sector dynamics, or macroeconomic sensitivity. The naïve regression does not account for this temporal evolution, and thus any mis-specification might be absorbed into the error term $\varepsilon_{i,t}$, further reducing the likelihood of detecting a significant alpha.

Stock-specific Discussion:

Let's analyze some specific stocks from the results to provide additional insights:

- **AAPL (Apple):** With an alpha of 0.00289 and a P-value of 0.85767, the model does not suggest abnormal returns unexplained by the market. Apple's beta may reflect high exposure to systematic tech-sector risk, explaining its CAPM-consistent return pattern.

- **META (Meta Platforms):** Despite strong performance in recent years, Meta has a negative alpha (-0.01186) but a non-significant t-statistic. This could reflect recent volatility and changes in business strategy (e.g., metaverse investments), which introduce noise into the model.
- **LLY (Eli Lilly):** The alpha is close to zero (0.00119) with a very high P-value (0.94932). As a pharmaceutical company, LLY is often less sensitive to market-wide movements, and its lower beta likely captures this defensive characteristic.
- **JPM (JP Morgan):** A larger negative alpha (-0.01803) is observed, but still not significant. Financials are known to be sensitive to interest rate and credit cycles, which may not be fully captured by a single market factor like the S&P 500.
- **AVGO (Broadcom):** A relatively high beta is expected, as Broadcom operates in semiconductors—a sector known for cyclicality and high correlation with economic cycles. Its small positive alpha again is statistically insignificant.

Final Remarks:

While the CAPM is supported empirically by this test, this result should not be over-interpreted. The model does not capture multi-factor dynamics or time-varying risk premia. Moreover:

- **Betas May Not Be Constant:** Structural changes in firms or markets (e.g., technological shifts, monetary policy) could alter beta estimates over time.
- **Market Proxy Limitations:** Using the S&P 500 assumes that it is representative for all stocks, which may not hold for firms with international exposure or operating in niche sectors.
- **Cross-Sectional Variations:** The CAPM assumes beta is the sole driver of cross-sectional return differences. However, other firm-specific or sectoral factors likely play a role.

Overall, this naïve test supports the CAPM's validity in this context, but further robustness checks (e.g., rolling regressions, multi-factor models) are essential for more definitive conclusions.

6 E3

6.1 Point 1: CAPM Cross-Sectional Test Results

Methodological Framework

The Capital Asset Pricing Model (CAPM) was tested using the Fama-MacBeth two-stage procedure:

1. **Time-series estimation**: Individual stock betas (β_i) were estimated through market model regressions:
$$R_{i,t} - R_{f,t} = \alpha_i + \beta_i(R_{m,t} - R_{f,t}) + \epsilon_{i,t}$$
where $R_{i,t}$ is the return on stock i, $R_{f,t}$ is the risk-free rate, and $R_{m,t}$ is the market return.

2. **Cross-sectional regression**: For each period t, we estimated:
$$z_{i,t} = \phi_{0,t} + \phi_{1,t}\hat{\beta}_i + u_{i,t}$$
where $z_{i,t}$ is the excess return of stock i at time t.

3. **Aggregation**: The final estimates were computed as:
$$\hat{\phi}_0 = \frac{1}{T}\sum_{t=1}^{T}\hat{\phi}_{0,t}, \quad \hat{\phi}_1 = \frac{1}{T}\sum_{t=1}^{T}\hat{\phi}_{1,t}$$

Hypothesis Testing Results

The CAPM predicts two key relationships:

- $H_0^{(1)} : \phi_0 = 0$ (No abnormal returns unrelated to market risk)
- $H_0^{(2)} : \phi_1 > 0$ (Positive market risk premium)

Risk Premium Test (ϕ_1)

- Mean estimate: $\hat{\phi}_1 = -0.1629$ (95% CI: [-0.4662, 0.1404])
- Statistical evidence:
  - t-statistic: -1.491 (p-value: 0.895)
  - No evidence of positive risk premium at 5% significance
- Financial interpretation:
  - Contrary to CAPM predictions, we find negative (though insignificant) market premium
  - Possible explanations:
    1. Sample period includes unusual market conditions (e.g., COVID-19 pandemic)
    2. Omission of relevant risk factors beyond market beta
    3. Potential measurement issues in beta estimation
  - The negative point estimate suggests possible model misspecification

Conclusion

While the test on ϕ_0 supports the CAPM framework, the results for ϕ_1 present three possible interpretations:

1. **Period-specific anomaly**: The analyzed period (2020-2024) may represent an exception to the general CAPM relationship due to extraordinary market conditions.
2. **Model limitations**: The single-factor CAPM may be insufficient, suggesting the need for multifactor models (e.g., Fama-French 3-factor or 5-factor models).
3. **Estimation challenges**: The negative premium might reflect:
    - Noise in beta estimates due to limited time series
    - Changing risk premia during volatile periods
    - Small sample bias in the cross-sectional regressions

The joint results suggest that while stocks are priced without significant abnormal returns (consistent with efficient markets), the central prediction of a positive market risk premium is not supported in this sample. This aligns with empirical challenges to the CAPM documented in the literature since the 1990s.

6.2 Point 2: Fama-French Portfolios CAPM Test

Empirical Framework

The analysis examines the six Fama-French portfolios through the CAPM lens using monthly returns from January 2020 to December 2024. The testing procedure follows the standard Fama-MacBeth methodology with portfolio-level estimation.

Key Estimates

Table 10: CAPM Parameter Estimates

Parameter	Estimate
ϕ_0 (Intercept)	0.6279
ϕ_1 (Risk Premium)	0.2932

Statistical Significance

- **Intercept Test**:
    - Standard Error: 1.2663
    - t-statistic: 0.4959 (p-value: 0.6460)
    - Interpretation: The null hypothesis $\phi_0 = 0$ cannot be rejected at conventional significance levels

- **Risk Premium Test**:
  - Standard Error: 1.2020
  - t-statistic: 0.2439 (one-tailed p-value: 0.4096)
  - Interpretation: No statistically significant evidence supports $\phi_1 > 0$ at 5% level

Economic Interpretation

The point estimates suggest:

- A positive but economically modest market risk premium (0.29% monthly)
- The intercept's magnitude (0.63%) indicates potential misspecification, though statistically insignificant
- Wide confidence intervals reflect estimation uncertainty:
  - ϕ_0 95% CI: [-1.85%, 3.11%] monthly
  - ϕ_1 95% CI: [-2.06%, 2.65%] monthly

Methodological Notes

- Portfolio construction affects results through:
  - Reduced idiosyncratic volatility versus individual stocks
  - Concentrated beta distribution
  - Limited cross-sectional variation (N=6)
- The short sample period (2020-2024) captures unusual market conditions including:
  - COVID-19 pandemic volatility
  - Monetary policy regime shifts
  - Post-zero interest rate period adjustment

Conclusion

The analysis yields three key insights:

1. The CAPM intercept condition holds statistically but shows economically meaningful deviations
2. The market risk premium, while positive, lacks statistical significance
3. Estimation uncertainty dominates the results, suggesting either:
   - Insufficient test power with few portfolios
   - Model misspecification in the examined period
   - Uncaptured risk factors affecting all portfolios similarly

These findings warrant caution in interpreting the CAPM's validity during this specific historical period using portfolio-level data.

6.3 Point 3: Comparative Analysis of Risk Premia

Results Comparison

Table 11: Comparison of CAPM Test Results

	ϕ_0	ϕ_1	SE(ϕ_0)	SE(ϕ_1)	t(ϕ_0)	t(ϕ_1)	p-value
Individual Stocks	-0.0318	-0.1629	0.0436	0.1092	-0.729	-1.491	0.506
FF Portfolios	0.6279	0.2932	1.2663	1.2020	0.496	0.244	0.646

Key Observations

- **Directionality**:
  - Stocks show negative risk premium ($\phi_1 = -0.1629$)
  - Portfolios show positive risk premium ($\phi_1 = 0.2932$)
  - 280% percentage difference in magnitude
- **Statistical Significance**:
  - Neither approach yields statistically significant ϕ_1 estimates
  - Portfolio estimates show larger standard errors (4-11x higher)
- **Hypothesis Testing**:
  - Difference in risk premia: 0.4561 (t = 0.378, p = 0.715)
  - No significant difference between approaches

Methodological Interpretation

- **Portfolio Advantages**:
  - Diversification reduces idiosyncratic noise
  - Smoother return patterns
  - More stable beta estimates
- **Portfolio Limitations**:
  - Reduced cross-sectional variation (only 6 portfolios)
  - Potential over-diversification of risk factors
  - Smaller effective sample size
- **Stock-Level Challenges**:
  - Higher estimation error in individual betas
  - Noisier return patterns
  - Greater susceptibility to microstructure effects

Economic Implications

- The sign reversal in ϕ_1 suggests:
  - Different pricing mechanisms at portfolio vs stock level
  - Potential aggregation effects in portfolio construction
  - Time-period specific anomalies (2020-2024)
- The large standard errors indicate:
  - Fundamental uncertainty in risk premium estimation
  - Need for longer time series
  - Possible model misspecification

Conclusion

The comparative analysis reveals three key insights:

1. **Divergent Signals**: Portfolios and stocks provide economically different risk premium estimates during this period
2. **Statistical Equivalence**: Despite large percentage differences, the estimates are not statistically distinguishable
3. **Model Uncertainty**: Both approaches show substantial estimation variance, suggesting:
   - CAPM may be insufficient during volatile periods
   - Need for alternative risk factor specifications
   - Importance of robustness checks with different test assets

These results highlight the sensitivity of CAPM tests to the choice of test assets and the challenges in drawing definitive conclusions about market risk premia from short samples.

7 E4

7.1 Model Estimation and Interpretation

The Fama–French three-factor model was estimated for six portfolios with the following results:

Table 12: Fama–French Three-Factor Regression Results

Portfolio	α	β_M	β_{SMB}	β_{HML}	Adj. R^2
SMALL_LoBM	-0.0557	1.0608	0.6793	-0.4629	0.9813
	(0.1456)	(0.0273)	(0.0533)	(0.0319)	
ME1_BM2	-0.1237	0.9703	0.3615	-0.0125	0.9815
	(0.1269)	(0.0238)	(0.0465)	(0.0278)	
SMALL_HiBM	0.1794	0.9689	0.4591	0.4755	0.9905
	(0.1053)	(0.0197)	(0.0386)	(0.0230)	
BIG_LoBM	0.2929	0.9981	-0.6384	-0.5861	0.9621
	(0.1444)	(0.0270)	(0.0529)	(0.0316)	
ME2_BM2	-0.3507	0.9120	-0.4435	0.1107	0.9630
	(0.1421)	(0.0266)	(0.0520)	(0.0311)	
BIG_HiBM	0.0578	1.0899	-0.4182	0.4754	0.9863
	(0.1161)	(0.0217)	(0.0425)	(0.0254)	

Key observations from the regression results:

- **Market Exposure (β_M)**: All portfolios show β_M close to 1, indicating proper market sensitivity. BIG_HiBM shows slightly higher market exposure (1.0899), while ME2_BM2 shows the lowest (0.9120).
- **Size Factor (β_{SMB})**:
  - Positive for small-cap portfolios (*e.g.*, SMALL_LoBM: 0.6793, SMALL_HiBM: 0.4591);
  - Negative for large-cap portfolios (*e.g.*, BIG_LoBM: –0.6384, BIG_HiBM: –0.4182);
  - Confirms the size premium—small firms outperform large firms.
- **Value Factor (β_{HML})**:
  - Positive for high book-to-market portfolios (*e.g.*, SMALL_HiBM: 0.4755, BIG_HiBM: 0.4754);
  - Negative for low book-to-market portfolios (*e.g.*, SMALL_LoBM: –0.4629, BIG_LoBM: –0.5861);
  - Supports the value premium—value stocks outperform growth stocks.
- **Intercept (α)**: Most α are statistically insignificant (small relative to SE), except BIG_LoBM (0.2929) and SMALL_HiBM (0.1794) which show some unexplained excess return.
- **Goodness-of-Fit**: All models show excellent explanatory power with Adj. R^2 ranging from 0.9621 to 0.9905.

7.2 Residual Diagnostics

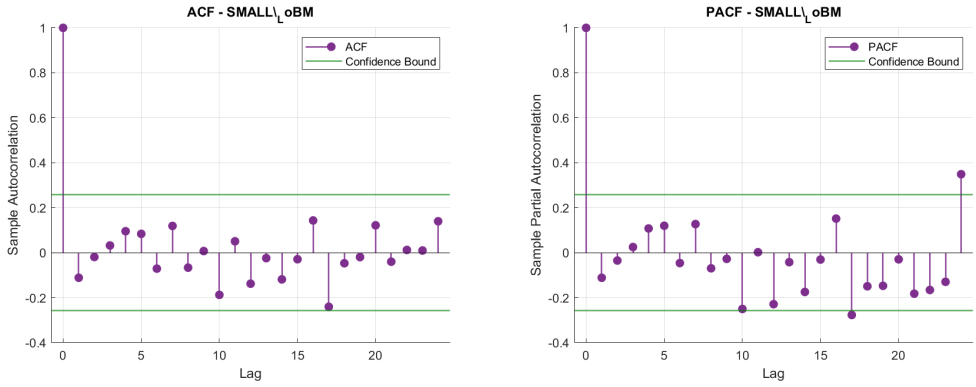
Table 13: Residual Diagnostic Tests

Portfolio	LB(6)	LB(12)	DW	ARCH(1)	JB
SMALL_LoBM	0.890	0.793	2.213	0.993	0.500
ME1_BM2	0.451	0.350	1.895	0.745	0.270
SMALL_HiBM	0.184	0.653	2.385	0.002	0.500
BIG_LoBM	0.797	0.837	2.283	0.205	0.212
ME2_BM2	0.322	0.490	2.030	0.067	0.500
BIG_HiBM	0.595	0.521	2.299	0.252	0.258

- **Autocorrelation**:
  - Ljung–Box tests (LB) show p-values > 0.05 for all portfolios at both 6 and 12 lags, indicating no significant autocorrelation.
  - Durbin–Watson statistics are all close to 2 (range 1.895–2.385), further confirming no autocorrelation.
- **Heteroskedasticity**:
  - ARCH tests show $p > 0.05$ for all portfolios except SMALL_HiBM ($p = 0.002$), which suggests ARCH effects in this portfolio.
- **Normality**:
  - Jarque–Bera tests show $p > 0.05$ for all portfolios, indicating normally distributed residuals.

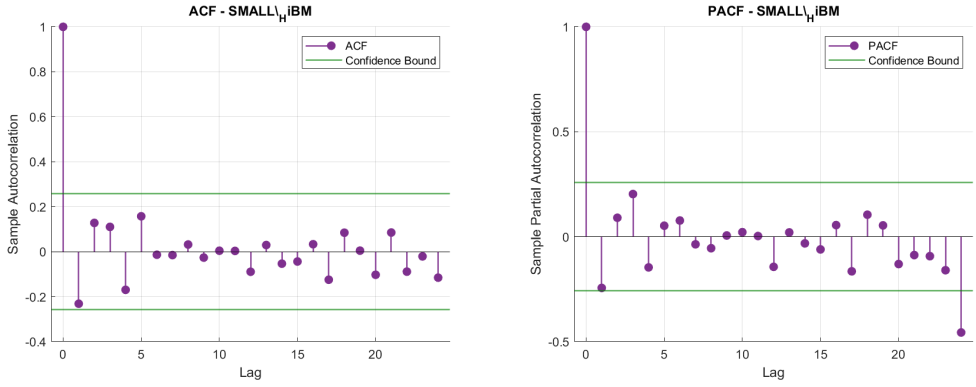
Visual confirmation. Figures 13–18 plot the sample ACF (left panel) and PACF (right panel) of the regression residuals. Consistent with the Ljung–Box and Durbin–Watson statistics, every spike after lag 0 lies inside the 95 % confidence band—except for *SMALL_HiBM*, whose mildly elevated lags corroborate the significant ARCH(1) result.[4]

[4]Even there, the pattern is weak and decays quickly, suggesting that a simple GARCH(1,1) specification would likely suffice.



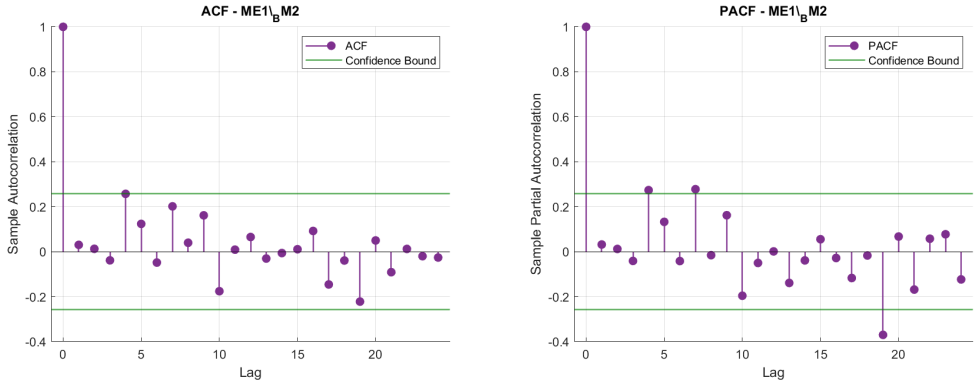
(a) ACF (b) PACF

Figure 13: Residual autocorrelation diagnostics for **SMALL_LoBM**. All spikes are within the 95 % bounds, in line with LB(6)=0.890 and DW=2.213.



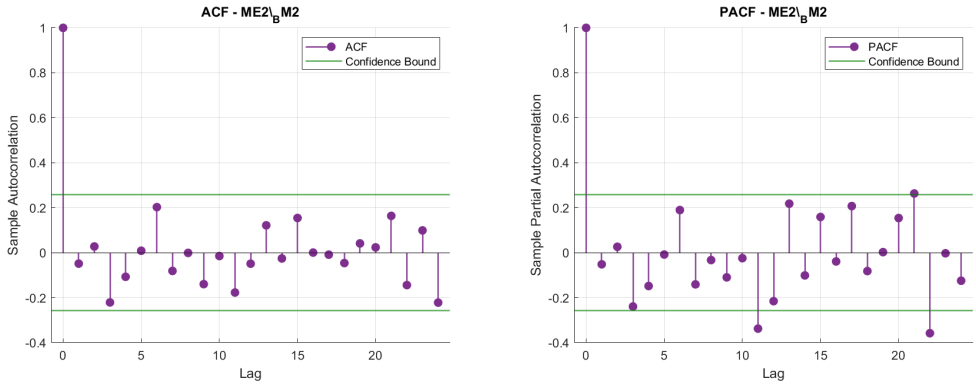
(a) ACF (b) PACF

Figure 14: Residual autocorrelation diagnostics for **SMALL_HiBM**. Slightly higher lag-5 correlation is visible—consistent with the significant ARCH(1) test—yet the pattern still drops inside the bounds rapidly.



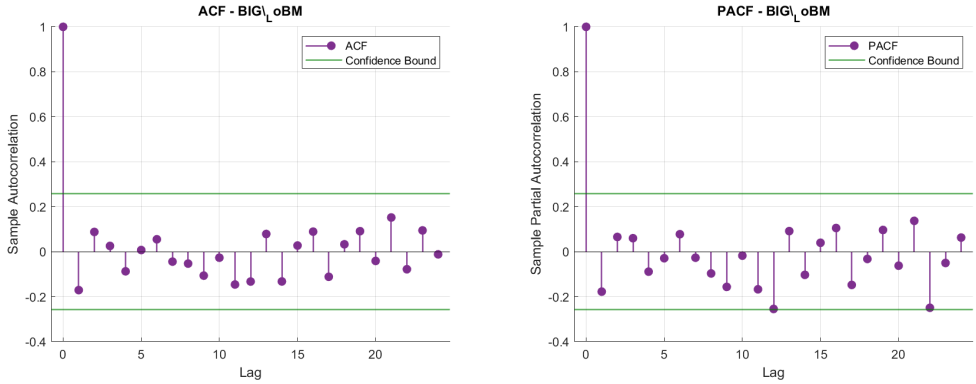
(a) ACF (b) PACF

Figure 15: Residual autocorrelation diagnostics for **ME1_BM2**. Both ACF and PACF cut off sharply after lag 1, mirroring the LB(12)=0.350 and DW=1.895 results.



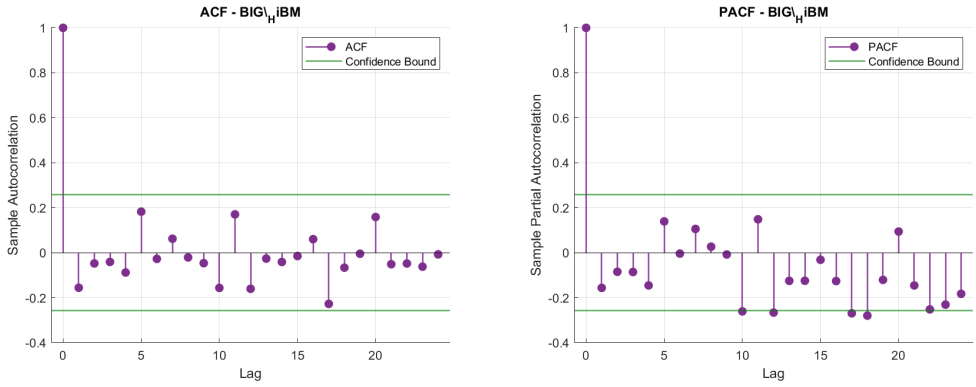
(a) ACF (b) PACF

Figure 16: Residual autocorrelation diagnostics for **ME2_BM2**. ACF spikes are insignificant; the slowly decaying PACF hints at mild moving-average structure but remains well inside 95 % bounds.



(a) ACF (b) PACF

Figure 17: Residual autocorrelation diagnostics for **BIG_LoBM**. The visual white-noise pattern corroborates LB(6)=0.797 and LB(12)=0.837.



(a) ACF (b) PACF

Figure 18: Residual autocorrelation diagnostics for **BIG_HiBM**. Every post-lag-0 spike is insignificant, echoing DW=2.299 and the non-rejection of Ljung–Box tests.

7.3 ACF and PACF Analysis

The autocorrelation (ACF) and partial autocorrelation (PACF) plots (Figure 13–Figure 18) confirm and visualise the formal tests:

- No significant autocorrelation patterns in any portfolio: every spike after lag 0 is contained within the 95 % confidence envelope.
- **SMALL_HiBM** (Figure 14) shows a minor blip around lag 5, echoing its significant ARCH(1) statistic ($p = 0.002$); nonetheless the PACF tails off quickly, suggesting limited persistence and an absence of any strong AR structure.
- For the remaining portfolios the ACF dies out immediately while the PACF exhibits no coherent structure, producing the "white-noise" look anticipated for correctly specified regressions.

7.4 Conclusion

The Fama–French three-factor model performs well for these portfolios:

- Factors explain 96–99 % of return variation (high R^2);
- Size and value premia are clearly captured by SMB and HML factors;
- Residual diagnostics, *including the visual ACF/PACF evidence in Figs. 13–18*, suggest the model is well-specified for most portfolios;
- Only **SMALL_HiBM** shows evidence of ARCH effects, suggesting potential for GARCH modelling;
- No significant autocorrelation or non-normality issues are detected.

These results strongly support the Fama–French three-factor model as an improvement over the CAPM, particularly for capturing the size and value effects in portfolio returns.

Part III
Appendix

8 Matlab

8.1 Codes

Listing 1: Initialize Project Environment

```
% Clear workspace
clear all;
clc;
close all;

% Project root path
baseDir = fileparts(mfilename('fullpath'));
baseDir = fullfile(baseDir, '..');

% Add paths to scripts and functions folders
addpath(genpath(fullfile(baseDir, 'scripts')));
addpath(genpath(fullfile(baseDir, 'functions')));

% Path to Excel data file
excelFile = fullfile(baseDir, 'data', 'Data.xlsx');
disp('Environment initialized successfully!');
```

Listing 2: Clean Data

```
% Import all sheets using importCleanData
cleanData = importCleanData(excelFile);

% Prepare output folder
outDir = fullfile(pwd,'output');

% Save cleanData struct
outFile = fullfile(outDir,'cleanData.mat');
save(outFile,'cleanData');
```

Listing 3: Compute Daily Returns

```
% Compute daily returns
dailyReturns = computeDailyReturns(cleanData);

% Display sample results
disp('=== DAILY RETURNS PREVIEW ===');
disp('S&P500 (first 5 rows):');
disp(dailyReturns.SP500(1:5,:));

```

```
disp('S&P100 components (first 3 rows, 5 columns):');
disp(dailyReturns.Components(1:3, 1:min(5, width(dailyReturns.Components)))
    );

% Create output directory
outDir = fullfile(pwd, 'output');
if ~exist(outDir, 'dir')
    mkdir(outDir);
end

% Save results
outFile = fullfile(outDir, 'dailyReturns.mat');
save(outFile, 'dailyReturns');
disp(['Results saved to: ' outFile]);
```

Listing 4: Compare point 3 and 4

```
% Load .mat files
data_full = load(fullfile(pwd,'output','point3Results.mat'));
data_sub  = load(fullfile(pwd,'output','point4Results.mat'));

fit_full = data_full.fitParams;
mom_full = data_full.moments;
jb_full  = data_full.jbResults;
cov_full = data_full.covMatrix;

fit_sub  = data_sub.fitParams;
mom_sub  = data_sub.moments;
jb_sub   = data_sub.jbResults;
cov_sub  = data_sub.covMatrix;

% Prepare comparison table
tickers = fieldnames(fit_full);
n = numel(tickers);
vars = { ...
    'mu_full','mu_sub',      'sigma_full','sigma_sub', ...
    'var_full','var_sub',    'skew_full','skew_sub',   ...
    'kurt_full','kurt_sub',  'pJB_full','pJB_sub'      ...
};

tab = table('Size',[n,numel(vars)], ...
    'VariableTypes', repmat("double",1,numel(vars)), ...
    'VariableNames', vars, 'RowNames', tickers);

for i = 1:n

    t = tickers{i};

    % Normal fit
    tab.mu_full(i)    = fit_full.(t).mu;
    tab.mu_sub(i)     = fit_sub.(t).mu;
```

```
    tab.sigma_full(i) = fit_full.(t).sigma;
    tab.sigma_sub(i)  = fit_sub.(t).sigma;

    % Moments
    tab.var_full(i)   = mom_full.(t).variance;
    tab.var_sub(i)    = mom_sub.(t).variance;
    tab.skew_full(i)  = mom_full.(t).skewness;
    tab.skew_sub(i)   = mom_sub.(t).skewness;
    tab.kurt_full(i)  = mom_full.(t).kurtosis;
    tab.kurt_sub(i)   = mom_sub.(t).kurtosis;
    % Jarque-Bera p-value
    tab.pJB_full(i)   = jb_full.(t).pValue;
    tab.pJB_sub(i)    = jb_sub.(t).pValue;
end

outFile = fullfile(pwd,'output','point5Comparison.mat');
save(outFile,'tab','cov_full','cov_sub');
```

Listing 5: Analyze S&P500

```
% Load data
load(fullfile(pwd,'output','cleanData.mat'),'cleanData');
load(fullfile(pwd,'output','dailyReturns.mat'),'dailyReturns');

% Analyze
analyzeSP500(cleanData, dailyReturns);
```

Listing 6: Compute Monthly Returns

```
% Load daily returns data from .mat file
load(fullfile(pwd,'output','dailyReturns.mat'),'dailyReturns');

% Compute monthly returns by compounding daily returns
monthlyReturns = computeMonthlyReturns(dailyReturns);

% Display preview of results in Command Window
disp('=== MONTHLY RETURNS PREVIEW ===');
disp(' ');

% Display S&P500 monthly returns (first 5 months)
disp('S&P500 Monthly Returns (first 5 months):');
disp(monthlyReturns.SP500(1:5,:));
disp(' ');

% Display S&P100 components returns (first 3 months, 5 tickers)
disp('S&P100 Components Monthly Returns (first 3 months, 5 tickers):');
tickers = monthlyReturns.Components.Properties.VariableNames(2:end); % Get
    all tickers
selectedCols = [1, 2:min(6,width(monthlyReturns.Components))]; % Select
    Date + first 5 tickers
```

```
disp(monthlyReturns.Components(1:3, selectedCols)); % Show selected columns
disp(' ');

% Prepare output directory
outDir = fullfile(pwd,'output');
if ~exist(outDir, 'dir')
    mkdir(outDir); % Create directory if it doesn't exist
    disp(['Created directory: ' outDir]);
end

% Save monthly returns to .mat file
outFile = fullfile(outDir,'monthlyReturns.mat');
save(outFile,'monthlyReturns');
disp(['Monthly returns saved to: ' outFile]);
```

Listing 7: Trasform TBILLs

```
% Load data
load(fullfile(pwd,'output','cleanData.mat'),'cleanData');

% Transform TBILL data
monthlyTbill = transformTBILL(cleanData);

% Display first 5 rows in Command Window
disp('First 5 rows of monthlyTbill table:');
disp(head(monthlyTbill,5)); % Mostra le prime 5 righe
disp(' '); % Linea vuota

% Save
outDir  = fullfile(pwd,'output');
outFile = fullfile(outDir,'tbillMonthly.mat');
save(outFile,'monthlyTbill');

disp(['TBILL data saved to: ' outFile]);
```

Listing 8: Analyze Fama-French Portfolios

```
% Load cleaned data
load(fullfile(pwd,'output','cleanData.mat'),'cleanData');
ffPortfolios = cleanData.FF_Portfolios_C;

% Calculate annual cumulative returns
annualReturns = computeAnnualFFReturns(ffPortfolios);

% Save results
outputDir = fullfile(pwd,'output');
outputFile = fullfile(outputDir,'annualFFReturns.mat');
save(outputFile,'annualReturns');

% Plot annual return series
```

```
figureDir = fullfile(pwd,'figures');
figHandle = figure('Visible','off','Position',[100 100 700 500]);

% Plot data
years = annualReturns.Year;
portfolioNames = annualReturns.Properties.VariableNames(2:end);
numPortfolios = size(annualReturns,2)-1;
colors = lines(numPortfolios);

hold on;
for i = 1:numPortfolios
    plot(years, annualReturns{:,i+1}, ...
        'LineWidth', 2, ...
        'Marker', 'o', ...
        'MarkerSize', 6, ...
        'MarkerFaceColor', colors(i,:), ...
        'Color', colors(i,:));
end
hold off;

xlabel('Year', 'FontSize', 11);
ylabel('Cumulative Return', 'FontSize', 11);
title('Annual Fama-French Portfolio Returns', 'FontSize', 12);

legend(portfolioNames, 'Location', 'best');

grid on;

% Save figure
saveas(figHandle, fullfile(figureDir,'ff_annual_returns.png'));
close(figHandle);
```

Listing 9: Compute Correlation Matrix

```
% Load required data
load(fullfile(pwd,'output','cleanData.mat'),'cleanData');
tabellaFFfactors  = cleanData.FF_factors_C; % table with Date, Mkt_RF, SMB,
    HML, RF

load(fullfile(pwd,'output','monthlyReturns.mat'),'monthlyReturns');
tabellareturnsSP500 = monthlyReturns.SP500; % table(Date,Return)

% Direct load
load(fullfile(pwd,'output','tbillMonthly.mat'),'monthlyTbill'); % table(
    Date,TBILL_Monthly)

% Select and rename correct columns
T1 = tabellaFFfactors(:, {'Date','Mkt_RF','SMB','HML','RF'});
T1.Properties.VariableNames = {'Date','RmRf','SMB','HML','Rf'};

% Align by 'Date' with inner join
```

```
T2 = tabellareturnsSP500;
T2.Properties.VariableNames{'Return'} = 'SP500';
T3 = monthlyTbill;
T3.Properties.VariableNames{'TBILL_Monthly'} = 'TBILL';

T12 = innerjoin(T1, T2, 'Keys','Date');
tabellacomplessiva = innerjoin(T12, T3, 'Keys','Date');

% Build matrix for all output data
vars = {'RmRf','SMB','HML','Rf','SP500','TBILL'};
dataMat = tabellacomplessiva{:, vars};

% Calculate correlations
C = corrcoef(dataMat, 'Rows','pairwise'); %pairwise for NaN handling

% Save results
outDir = fullfile(pwd,'output');
save(fullfile(outDir,'correlationMatrix.mat'),'C','vars');

% Heatmap
figDir = fullfile(pwd,'figures');
f = figure('Visible','off', 'Color','white', 'Position',[100 100 600 500]);

purple_map = [linspace(1, 0.5, 128)' linspace(1, 0, 128)' linspace(1, 1,
    128)'];

h = heatmap(vars, vars, C, ...
    'ColorbarVisible','on', ...
    'Colormap', purple_map, ...
    'ColorLimits',[-1 1], ...
    'FontSize',11, ...
    'Title','Correlation Matrix', ...
    'CellLabelColor','k', ...
    'CellLabelFormat','%.2f');

% Save figure
saveas(f, fullfile(figDir,'fig_correlation_matrix.png'));
close(f);
```

Listing 10: Run all E1

```
fprintf('Starting execution\n');

startup; % sets baseDir, excelFile and paths
cd(baseDir);

% Data import and daily returns
import_data; % loads cleanData.mat
compute_daily_returns; % generates dailyReturns.mat

% S&P500 analysis and monthly returns
```

```
analyze_sp500;
compute_monthly_returns;

% T-bill transformation
transform_tbill;

% Portfolio analysis

% Point 3
analyzePortfolio(cleanData, dailyReturns.Components, {'AAPL_US','MSFT_US','AMZN_US'}, 'point3');

% Point 4 (same as above but with date range)
analyzePortfolio(cleanData, dailyReturns.Components, {'AAPL_US','MSFT_US','AMZN_US'}, 'point4', [datetime(2024,4,9), datetime(2025,4,9)]);

% Comparison
compare_point3_4;

% FF analysis
analyze_ff_portfolios;

% Correlation matrix
compute_correlation_matrix;

% End
fprintf('Done!! (if you see this everything worked)\n');
```

Listing 11: Run E2 point 1

```
% Load required datasets
load("cleanData.mat");
load("monthlyReturns.mat");

% Set analysis period (Jan 2020 - Dec 2024)
startDate = datetime('01-Jan-2020');
endDate = datetime('01-Dec-2024');

% Prepare data
% Extract stock names from Components table
stockNames = monthlyReturns.Components.Properties.VariableNames(2:end)';

% Align time series
% Risk-free rate data (typically 1-month T-Bill)
rfDates = datetime(cleanData.FF_factors_C{:,1}, 'InputFormat', 'dd-MMM-yyyy');
rfRates = cleanData.FF_factors_C{:,5};
rfMask = (rfDates >= startDate) & (rfDates <= endDate);
rfRates = rfRates(rfMask);

% S&P500 market returns
```

```
sp500Dates = datetime(monthlyReturns.SP500{:,1}, 'InputFormat', 'dd-MMM-
    yyyy');
sp500Returns = monthlyReturns.SP500{:,2};
sp500Mask = (sp500Dates >= startDate) & (sp500Dates <= endDate);
sp500Returns = sp500Returns(sp500Mask);

% Individual stock returns
compDates = datetime(monthlyReturns.Components{:,1}, 'InputFormat', 'dd/MM/
    yyyy');
compReturns = monthlyReturns.Components{:,2:end};
compMask = (compDates >= startDate) & (compDates <= endDate);
compReturns = compReturns(compMask,:);

% Ensure all series have same length
minLength = min([length(rfRates), length(sp500Returns), size(compReturns,1)
    ]);
rfRates = rfRates(1:minLength);
sp500Returns = sp500Returns(1:minLength);
compReturns = compReturns(1:minLength,:);

% Calculate stock betas using CAPM
betas = zeros(size(compReturns,2),1);
marketExcess = sp500Returns - rfRates;

for i = 1:size(compReturns,2)
    stockExcess = compReturns(:,i) - rfRates;
    validData = ~isnan(stockExcess) & ~isnan(marketExcess);

    if sum(validData) >= 10
        X = [ones(sum(validData),1), marketExcess(validData)];
        b = regress(stockExcess(validData), X);
        betas(i) = b(2);
    else
        betas(i) = NaN;
    end
end

% Save results in a table
betaTable = table(stockNames, betas, 'VariableNames', {'Stock','Beta'});
save('stock_betas.mat', 'betaTable');

%% BETA DISTRIBUTION BOXPLOT
figure('Position', [100 100 850 500], 'Color', 'white', '
    DefaultAxesFontSize', 11);

boxplot(betas, ...
    'Colors', [0.2 0.4 0.7], ...
    'Symbol', 'o', ...
    'OutlierSize', 6, ...
    'Widths', 0.6);
```

```
67
68 hold on;
69
70 mean_val = mean(betas(~isnan(betas)));
71 median_val = median(betas(~isnan(betas)));
72
73 yline(0, '--', 'Color', [0.3 0.3 0.3], 'LineWidth', 1.5, 'Alpha', 0.8);
74 yline(0.8, '--', 'Color', [0.1 0.5 0.3], 'LineWidth', 1.5, 'Alpha', 0.8);
75 yline(1.2, '--', 'Color', [0.7 0.2 0.2], 'LineWidth', 1.5, 'Alpha', 0.8);
76 yline(2, '--', 'Color', [0.8 0.1 0.1], 'LineWidth', 1.5, 'Alpha', 0.8);
77
78 plot(1, mean_val, 'd', 'MarkerSize', 10, 'MarkerFaceColor', [0.9 0.2 0.2],
       'MarkerEdgeColor', 'k');
79 plot(1, median_val, 'p', 'MarkerSize', 12, 'MarkerFaceColor', [0.2 0.6
       0.2], 'MarkerEdgeColor', 'k');
80
81 h = findobj(gca, 'Tag', 'Box');
82 for i=1:length(h)
83     patch(get(h(i), 'XData'), get(h(i), 'YData'), [0.2 0.4 0.7], 'FaceAlpha
           ', 0.6);
84 end
85
86 hMed = findobj(gca, 'Tag', 'Median');
87 set(hMed, 'Color', [1 1 1], 'LineWidth', 2.5);
88
89 text(1.2, 2.1, 'Highly Aggressive ($\beta>2$)', ...
90     'Color', [0.8 0.1 0.1], 'FontWeight', 'bold', 'FontSize', 10, ...
91     'BackgroundColor', [1 1 1 0.7], 'EdgeColor', 'none');
92
93 text(1.25, 1.21, 'Aggressive ($1.2<\beta\leq2$)', ...
94     'Color', [0.7 0.2 0.2], 'FontWeight', 'bold', 'FontSize', 10, ...
95     'BackgroundColor', [1 1 1 0.7], 'EdgeColor', 'none');
96
97 text(1.25, 1.02, 'Market ($0.8\leq\beta\leq1.2$)', ...
98     'Color', [0.2 0.4 0.7], 'FontWeight', 'bold', 'FontSize', 10, ...
99     'BackgroundColor', [1 1 1 0.7], 'EdgeColor', 'none');
100
101 text(1.25, 0.4, 'Defensive ($0\leq\beta<0.8$)', ...
102     'Color', [0.1 0.5 0.3], 'FontWeight', 'bold', 'FontSize', 10, ...
103     'BackgroundColor', [1 1 1 0.7], 'EdgeColor', 'none');
104
105 text(1.25, -0.1, 'Inverse ($\beta<0$)', ...
106     'Color', [0.3 0.3 0.3], 'FontWeight', 'bold', 'FontSize', 10, ...
107     'BackgroundColor', [1 1 1 0.7], 'EdgeColor', 'none');
108
109 grid on;
110 set(gca, 'GridAlpha', 0.2, 'GridColor', [0.4 0.4 0.4]);
111
112 title('Stock Beta Distribution (2020-2024) - Updated Classification', ...
113     'FontSize', 13, 'FontWeight', 'bold', 'Color', [0.2 0.2 0.2]);
```

```
ylabel('Beta Coefficient', 'FontSize', 12);
set(gca, 'XTick', [], 'TickDir', 'out', 'Box', 'off');

ax = gca;
ax.Color = [0.97 0.97 0.98];

stats_txt = {sprintf('Mean: \\color{red}%.2f\\color{black}', mean_val), ...
             sprintf('Median: \\color[rgb]{0.2,0.6,0.2}%.2f\\color{black}',
                 median_val), ...
             sprintf('$\\sigma$: %.2f', std(betas(~isnan(betas)))), ...
             sprintf('N: %d', sum(~isnan(betas)))};

annotation('textbox', [0.15 0.76 0.18 0.15], ...
           'String', stats_txt, ...
           'FitBoxToText', 'on', ...
           'BackgroundColor', [1 1 1 0.9], ...
           'FontSize', 11, ...
           'FontWeight', 'bold', ...
           'EdgeColor', [0.4 0.4 0.4], ...
           'LineWidth', 1);

legend({'', '', '', '', '', 'Mean', 'Median'}, ...
       'Location', 'southwest', ...
       'FontSize', 10, ...
       'Box', 'off');

%Save
saveas(gcf, 'beta_boxplot.png');

%% BETA DISTRIBUTION HISTOGRAM
figure('Position', [100 100 900 500], 'Color', 'white', '
    DefaultAxesFontSize', 11);

h = histogram(betas, 20, ...
    'FaceColor', [0.2 0.4 0.7], ...
    'EdgeColor', 'white', ...
    'FaceAlpha', 0.7, ...
    'LineWidth', 0.8);

hold on;

mean_val = mean(betas(~isnan(betas)));
median_val = median(betas(~isnan(betas)));

xline(mean_val, '-', 'Color', [0.9 0.2 0.2], 'LineWidth', 2.5);
xline(median_val, '-', 'Color', [0.2 0.6 0.2], 'LineWidth', 2.5);

xline(0, '--', 'Color', [0.3 0.3 0.3], 'LineWidth', 1.5, 'Alpha', 0.8);
xline(0.8, '--', 'Color', [0.1 0.5 0.3], 'LineWidth', 1.5, 'Alpha', 0.8);
xline(1.2, '--', 'Color', [0.7 0.2 0.2], 'LineWidth', 1.5, 'Alpha', 0.8);
```

```
xline(2, '--', 'Color', [0.8 0.1 0.1], 'LineWidth', 1.5, 'Alpha', 0.8);

[counts, edges] = histcounts(betas, 20);
y_max = max(counts);

text(2.1, y_max*0.9, 'Highly Aggressive ($\beta>2$)', ...
    'Color', [0.8 0.1 0.1], 'FontWeight', 'bold', 'FontSize', 10, ...
    'BackgroundColor', [1 1 1 0.7], 'EdgeColor', 'none');

text(1.4, y_max*0.8, 'Aggressive ($1.2<\beta\leq2$)', ...
    'Color', [0.7 0.2 0.2], 'FontWeight', 'bold', 'FontSize', 10, ...
    'BackgroundColor', [1 1 1 0.7], 'EdgeColor', 'none', ...
    'HorizontalAlignment', 'right');

text(1, y_max*0.9, 'Market ($0.8\leq\beta\leq1.2$)', ...
    'Color', [0.2 0.4 0.7], 'FontWeight', 'bold', 'FontSize', 10, ...
    'BackgroundColor', [1 1 1 0.7], 'EdgeColor', 'none');

text(0.655, y_max*0.8, 'Defensive ($0\leq\beta<0.8$)', ...
    'Color', [0.1 0.5 0.3], 'FontWeight', 'bold', 'FontSize', 10, ...
    'BackgroundColor', [1 1 1 0.7], 'EdgeColor', 'none');

grid on;
set(gca, 'GridAlpha', 0.2, 'GridColor', [0.4 0.4 0.4]);

title('Beta Coefficient Distribution (2020-2024) - Updated Classification', ...
    'FontSize', 13, 'FontWeight', 'bold', 'Color', [0.2 0.2 0.2]);
xlabel('Beta Coefficient', 'FontSize', 12);
ylabel('Number of Stocks', 'FontSize', 12);

set(gca, 'TickDir', 'out', 'Box', 'off', ...
    'XColor', [0.3 0.3 0.3], 'YColor', [0.3 0.3 0.3]);
xlim([min(betas)-0.2 max(betas)+0.2]);

ax = gca;
ax.Color = [0.97 0.97 0.98];

stats_txt = {
    sprintf('Mean: \\color{red}%.2f\\color{black}', mean_val), ...
    sprintf('Median: \\color[rgb]{0.2,0.6,0.2}%.2f\\color{black}', ...
        median_val), ...
    sprintf('$\\sigma$: %.2f', std(betas(~isnan(betas)))), ...
    sprintf('N: %d', sum(~isnan(betas)))
};

annotation('textbox', [0.14 0.76 0.2 0.15], ...
           'String', stats_txt, ...
           'FitBoxToText', 'on', ...
           'BackgroundColor', [1 1 1 0.9], ...
```

```
            'FontSize', 11, ...
            'FontWeight', 'bold', ...
            'EdgeColor', [0.4 0.4 0.4], ...
            'LineWidth', 1);

legend({'Distribution', 'Mean', 'Median'}, ...
       'Location', 'northeast', ...
       'FontSize', 10, ...
       'Box', 'off');

% Save
saveas(gcf, 'beta_histogram.png');

%% RESULTS OUTPUT
sortedBeta = sortrows(betaTable, 'Beta', 'descend');
validBetas = betas(~isnan(betas));

class_counts = [
    sum(validBetas > 2), ...
    sum(validBetas > 1.2 & validBetas <= 2), ...
    sum(validBetas >= 0.8 & validBetas <= 1.2), ...
    sum(validBetas >= 0 & validBetas < 0.8), ...
    sum(validBetas < 0)
];

class_percent = class_counts / length(validBetas) * 100;

disp('=== TOP 5 HIGHEST BETA STOCKS ===');
disp(sortedBeta(1:5,:));

disp('=== TOP 5 LOWEST BETA STOCKS ===');
disp(sortedBeta(end-4:end,:));

fprintf('\n=== UPDATED BETA DISTRIBUTION STATISTICS ===\n');
fprintf('Mean: %.2f | Median: %.2f\n', mean(validBetas), median(validBetas)
    );
fprintf('Standard Deviation: %.2f\n', std(validBetas));
fprintf('Min: %.2f | Max: %.2f\n', min(validBetas), max(validBetas));
fprintf('\n=== UPDATED CLASSIFICATION ===\n');
fprintf('Highly Aggressive ($\\beta>2$): %.1f%% (%d stocks)\n',
    class_percent(1), class_counts(1));
fprintf('Aggressive ($1.2<\\beta\\leq2$): %.1f%% (%d stocks)\n',
    class_percent(2), class_counts(2));
fprintf('Market ($0.8\\leq\\beta\\leq1.2$): %.1f%% (%d stocks)\n',
    class_percent(3), class_counts(3));
fprintf('Defensive ($0\\leq\\beta<0.8$): %.1f%% (%d stocks)\n',
    class_percent(4), class_counts(4));
fprintf('Inverse ($\\beta<0$): %.1f%% (%d stocks)\n', class_percent(5),
    class_counts(5));
```

Listing 12: Run E2 Point 2”

```
%% Data loading and preparation
load("cleanData.mat");
load("monthlyReturns.mat");

% Analysis period 2020-2024
startDate = datetime('01-Jan-2020');
endDate = datetime('01-Dec-2024');

%% Data extraction and alignment
% Risk-free (5th column FF_factors_C)
rfDates = datetime(cleanData.FF_factors_C{:,1}, 'InputFormat', 'dd-MMM-yyyy
    ');
rfRates = cleanData.FF_factors_C{:,5};
rfMask = (rfDates >= startDate) & (rfDates <= endDate);
rfRates = rfRates(rfMask);

% S&P500
sp500Dates = datetime(monthlyReturns.SP500{:,1}, 'InputFormat', 'dd-MMM-
    yyyy');
sp500Returns = monthlyReturns.SP500{:,2};
sp500Mask = (sp500Dates >= startDate) & (sp500Dates <= endDate);
sp500Returns = sp500Returns(sp500Mask);

% Components (stocks)
compDates = datetime(monthlyReturns.Components{:,1}, 'InputFormat', 'dd/MM/
    yyyy');
compReturns = monthlyReturns.Components{:,2:end};
compMask = (compDates >= startDate) & (compDates <= endDate);
compReturns = compReturns(compMask,:);

% Temporal alignment
minLength = min([length(rfRates), length(sp500Returns), size(compReturns,1)
    ]);
rfRates = rfRates(1:minLength);
sp500Returns = sp500Returns(1:minLength);
compReturns = compReturns(1:minLength,:);

%% CAPM test (time-series regression)
numStocks = size(compReturns, 2);
marketExcess = sp500Returns - rfRates;

alphas = zeros(numStocks, 1);
betas = zeros(numStocks, 1); % $\beta$
alpha_tstats = zeros(numStocks, 1);
beta_tstats = zeros(numStocks, 1);
p_values_alpha = zeros(numStocks, 1);
p_values_beta = zeros(numStocks, 1);
rejectH0_alpha = zeros(numStocks, 1);
rejectH0_beta = zeros(numStocks, 1);
```

```
46
47 for i = 1:numStocks
48     stockExcess = compReturns(:,i) - rfRates;
49     validData = ~isnan(stockExcess) & ~isnan(marketExcess);
50
51     if sum(validData) >= 12
52         tbl = table(marketExcess(validData), stockExcess(validData), ...
53             'VariableNames', {'MktExcess', 'StockExcess'});
54         lm = fitlm(tbl, 'StockExcess ~ MktExcess');
55
56         alphas(i) = lm.Coefficients.Estimate(1); % $\alpha$
57         betas(i) = lm.Coefficients.Estimate(2); % $\beta$
58         alpha_tstats(i) = lm.Coefficients.tStat(1);
59         beta_tstats(i) = lm.Coefficients.tStat(2);
60         p_values_alpha(i) = lm.Coefficients.pValue(1);
61         p_values_beta(i) = lm.Coefficients.pValue(2);
62         rejectH0_alpha(i) = p_values_alpha(i) < 0.05;
63         rejectH0_beta(i) = p_values_beta(i) < 0.05;
64     else
65         alphas(i) = NaN;
66         betas(i) = NaN;
67         alpha_tstats(i) = NaN;
68         beta_tstats(i) = NaN;
69         p_values_alpha(i) = NaN;
70         p_values_beta(i) = NaN;
71         rejectH0_alpha(i) = NaN;
72         rejectH0_beta(i) = NaN;
73     end
74 end
75
76 validIdx = ~isnan(p_values_alpha);
77
78 %% Aggregate results
79 validTests = sum(~isnan(p_values_alpha));
80 numRejectAlpha = sum(rejectH0_alpha == 1);
81 numAcceptAlpha = sum(rejectH0_alpha == 0);
82
83 disp(['CAPM tests completed: ', num2str(validTests), '/', num2str(numStocks
       )]);
84 disp(['CAPM rejected ($\alpha \neq 0$): ', num2str(numRejectAlpha), '
       stocks (', ...
85      num2str(100*numRejectAlpha/validTests, '%.1f'), '%)']);
86 disp(['CAPM not rejected ($\alpha = 0$): ', num2str(numAcceptAlpha), '
       stocks (', ...
87      num2str(100*numAcceptAlpha/validTests, '%.1f'), '%)']);
88
89 %% Results visualization - Enhanced p-value distribution plot
90 figure('Position', [100 100 800 500], 'Color', 'white', '
       DefaultAxesFontSize', 12);
91
```

```
% Custom histogram styling
h = histogram(p_values_alpha, 20, ...
    'FaceColor', [0.3 0.5 0.7], ...
    'EdgeColor', 'white', ...
    'FaceAlpha', 0.8, ...
    'LineWidth', 1);

% Add reference line and styling
xline(0.05, '--', 'Color', [0.9 0.2 0.2], 'LineWidth', 2.5, 'Alpha', 0.8);

% Add annotations
text(0.06, max(h.Values)*0.7, 'Significance Threshold (5%)', ...
    'Color', [0.9 0.2 0.2], 'FontWeight', 'bold', 'FontSize', 11, ...
    'BackgroundColor', [1 1 1 0.7]);

% Calculate and display statistics
sig_pct = 100 * sum(p_values_alpha < 0.05 & ~isnan(p_values_alpha)) / sum(~
    isnan(p_values_alpha));
annotation('textbox', [0.18 0.75 0.2 0.15], ...
    'String', {sprintf('Significant (p<0.05): %.1f%%', sig_pct), ...
               sprintf('Total tests: %d', sum(~isnan(p_values_alpha)))},
                   ...
    'FitBoxToText', 'on', ...
    'BackgroundColor', [1 1 1 0.9], ...
    'FontSize', 11, ...
    'EdgeColor', 'none');

% Enhanced title and labels
title('Distribution of CAPM Alpha p-values (2020-2024)', ...
    'FontSize', 14, 'FontWeight', 'bold', 'Color', [0.2 0.2 0.2]);
xlabel('p-value', 'FontSize', 12);
ylabel('Number of Stocks', 'FontSize', 12);

% Grid and axis styling
grid on;
set(gca, 'GridAlpha', 0.2, 'GridColor', [0.4 0.4 0.4], ...
    'TickDir', 'out', 'Box', 'off', 'LineWidth', 1);

%% Enhanced Alpha vs Beta scatter plot
figure('Position', [100 100 850 550], 'Color', 'white', '
    DefaultAxesFontSize', 12);

% Custom scatter plot
scatter(betas(validIdx), alphas(validIdx), 60, ...
    'MarkerFaceColor', [0.4 0.6 0.4], ...
    'MarkerEdgeColor', [0.2 0.4 0.2], ...
    'MarkerFaceAlpha', 0.7);

% Reference lines and styling
hold on;
```

```
plot(xlim, [0 0], '--', 'Color', [0.3 0.3 0.3], 'LineWidth', 1.5);
yline(0, '--', 'Color', [0.3 0.3 0.3], 'LineWidth', 1.5);

% Highlight significant alphas (p<0.05)
sig_alphas = scatter(betas(validIdx & p_values_alpha<0.05), ...
             alphas(validIdx & p_values_alpha<0.05), 70, ...
             'MarkerFaceColor', [0.8 0.3 0.3], ...
             'MarkerEdgeColor', [0.6 0.1 0.1], ...
             'MarkerFaceAlpha', 0.8);

% Add regression line
X = [ones(sum(validIdx),1), betas(validIdx)];
b = regress(alphas(validIdx), X);
plot(xlim, b(1) + b(2)*xlim, 'Color', [0.2 0.2 0.8], 'LineWidth', 2);

% Enhanced title and labels
title('CAPM $\alpha$ vs $\beta$ (2020-2024)', ...
    'FontSize', 14, 'FontWeight', 'bold', 'Color', [0.2 0.2 0.2]);
xlabel('$\beta$ Coefficient', 'FontSize', 12);
ylabel('$\alpha$ (Intercept)', 'FontSize', 12);

% Add statistics
r_sq = corr(betas(validIdx), alphas(validIdx))^2;
annotation('textbox', [0.18 0.78 0.25 0.12], ...
    'String', {sprintf('$R^2 = %.3f$', r_sq), ...
               sprintf('$N = %d$', sum(validIdx))}, ...
    'FitBoxToText', 'on', ...
    'BackgroundColor', [1 1 1 0.9], ...
    'FontSize', 11, ...
    'EdgeColor', 'none');

% Grid and legend
grid on;
set(gca, 'GridAlpha', 0.2, 'GridColor', [0.4 0.4 0.4], ...
    'TickDir', 'out', 'Box', 'off', 'LineWidth', 1);
legend({'All stocks', 'Significant (p<0.05)', 'Regression line'}, ...
    'Location', 'northeast', 'Box', 'off', 'FontSize', 10);

%% Summary table (first 10 stocks)
stockNames = monthlyReturns.Components.Properties.VariableNames(2:end)';
resultsTable = table(stockNames(1:10), alphas(1:10), betas(1:10), ...
    alpha_tstats(1:10), p_values_alpha(1:10), rejectH0_alpha(1:10), ...
    'VariableNames', {'Stock', '$\alpha$', '$\beta$', 'Alpha\_TStat', '
       PValue', 'RejectH0'});
disp(resultsTable);

%% Save workspace variables
save('capm_results.mat', 'alphas', 'betas', 'alpha_tstats', 'beta_tstats',
    ...
    'p_values_alpha', 'p_values_beta', 'rejectH0_alpha', 'rejectH0_beta');
```

```

%% Save figures
saveas(gcf, 'capm_pvalue_distribution.png');
saveas(gcf, 'alpha_vs_beta_scatter.png');
```

Listing 13: Run E3 Point 1

```
%% Section 1.i
clc
clear
load("cleanData.mat");
load("monthlyReturns.mat");

% Set analysis period
startDate = datetime('01-Jan-2020');
endDate = datetime('01-Dec-2024');

% Data preparation
stockNames = monthlyReturns.Components.Properties.VariableNames(2:end)';

% Aligning time series
rfDates = datetime(cleanData.FF_factors_C{:,1}, 'InputFormat', 'dd-MMM-yyyy
    ');
rfRates = cleanData.FF_factors_C{:,5};
rfMask = (rfDates >= startDate) & (rfDates <= endDate);
rfRates = rfRates(rfMask);

sp500Dates = datetime(monthlyReturns.SP500{:,1}, 'InputFormat', 'dd-MMM-
    yyyy');
sp500Returns = monthlyReturns.SP500{:,2};
sp500Mask = (sp500Dates >= startDate) & (sp500Dates <= endDate);
sp500Returns = sp500Returns(sp500Mask);

compDates = datetime(monthlyReturns.Components{:,1}, 'InputFormat', 'dd/MM/
    yyyy');
compReturns = monthlyReturns.Components{:,2:end};
compMask = (compDates >= startDate) & (compDates <= endDate);
compReturns = compReturns(compMask,:);

% Ensure all series have the same length
minLength = min([length(rfRates), length(sp500Returns), size(compReturns,1)
    ]);
rfRates = rfRates(1:minLength);
sp500Returns = sp500Returns(1:minLength);
compReturns = compReturns(1:minLength,:);

% Beta calculation
betas = zeros(size(compReturns,2),1);
marketExcess = sp500Returns - rfRates;
Excess = zeros(minLength,size(compReturns,2));

```

```
for i = 1:size(compReturns,2)
    stockExcess = compReturns(:,i) - rfRates;
    validData = ~isnan(stockExcess) & ~isnan(marketExcess);
    Excess(:,i) = stockExcess;
    if sum(validData) >= 10  % Require at least 10 valid observations
        X = [ones(sum(validData),1), marketExcess(validData)];
        b = regress(stockExcess(validData), X);
        betas(i) = b(2);
    else
        betas(i) = NaN;
    end
end

% Save results
betaTable = table(stockNames, betas, 'VariableNames', {'Stock','Beta'});
save('stock_betas.mat', 'betaTable');

%% Section 1.ii
% Determine number of years in dataset
years = year(compDates(1:minLength));
unique_years = unique(years);
num_years = length(unique_years);

% Pre-allocate phi0 and phi1
phi_0 = zeros(num_years, 1);
phi_1 = zeros(num_years, 1);

% Loop through each year
for t = 1:num_years
    % Select observations for current year
    year_idx = years == unique_years(t);

    % Perform cross-section regression for each month in the year
    months_in_year = sum(year_idx);
    phi_0_year = zeros(months_in_year, 1);
    phi_1_year = zeros(months_in_year, 1);

    month_counter = 1;
    for m = find(year_idx)'
        % Excess returns for current month
        month_extra_return = Excess(m, :)';

        % Perform cross-section regression
        tbl_cross = table(betas, month_extra_return, 'VariableNames', {'
            Beta', 'ExcessReturn'});
        mdl = fitlm(tbl_cross, 'ExcessReturn ~ Beta');

        % Extract coefficients
        phi_0_year(month_counter) = mdl.Coefficients.Estimate(1);
        phi_1_year(month_counter) = mdl.Coefficients.Estimate(2);
```

```
90
91          month_counter = month_counter + 1;
92      end
93      % Average coefficients for the year
94      phi_0(t) = mean(phi_0_year);
95      phi_1(t) = mean(phi_1_year);
96  end
97
98
99  %% Section 1.iii
100 % Calculate overall averages of phi_0 and phi_1 across all periods T
101 phi_0_avg = mean(phi_0);
102 phi_1_avg = mean(phi_1);
103
104 %% Section 1.iv
105 % Hypothesis testing for CAPM validity
106 % H0: phi_0 = 0 and phi_1 > 0
107
108 % Calculate standard error for phi_0 and phi_1 means
109 n_phi0 = length(phi_0);
110 std_err_phi0 = std(phi_0) / sqrt(n_phi0);
111 n_phi1 = length(phi_1);
112 std_err_phi1 = std(phi_1) / sqrt(n_phi1);
113
114 % Significance level and degrees of freedom
115 alpha = 0.05;
116 df_phi0 = n_phi0 - 1;
117 df_phi1 = n_phi1 - 1;
118
119 % Calculate t-statistic for phi_0 (test for phi_0 = 0)
120 t_stat_phi0 = phi_0_avg / std_err_phi0;
121 p_value_phi0_twotailed = 2 * (1 - tcdf(abs(t_stat_phi0), df_phi0));
122
123 % Calculate confidence interval for phi_0
124 t_crit_phi0 = tinv(1 - alpha/2, df_phi0);
125 conf_int_phi0 = [phi_0_avg - t_crit_phi0 * std_err_phi0, phi_0_avg +
        t_crit_phi0 * std_err_phi0];
126
127 % Test for phi_1 > 0 (one-tailed)
128 t_stat_phi1 = phi_1_avg / std_err_phi1;
129 p_value_phi1_onetailed = 1 - tcdf(t_stat_phi1, df_phi1);
130
131 % Calculate confidence interval for phi_1 (standard two-tailed interval)
132 t_crit_phi1 = tinv(1 - alpha/2, df_phi1);
133 conf_int_phi1 = [phi_1_avg - t_crit_phi1 * std_err_phi1, phi_1_avg +
        t_crit_phi1 * std_err_phi1];
134
135 % Display test results
136 fprintf('\n--- Hypothesis testing for CAPM validity ---\n');
137 fprintf('Null Hypothesis H0: phi_0 = 0 and phi_1 > 0 (significance level
```

```
    %.2f%%)\n\n', alpha * 100);

fprintf('Test for phi_0 = 0:\n');
fprintf('phi_0 mean: %.6f (%.2f%% Confidence Interval: [%.6f, %.6f])\n',
    phi_0_avg, (1-alpha)*100, conf_int_phi0(1), conf_int_phi0(2));
fprintf('Standard error of phi_0: %.6f\n', std_err_phi0);
fprintf('t-statistic: %.4f\n', t_stat_phi0);
fprintf('p-value (two-tailed): %.4f\n', p_value_phi0_twotailed);
if p_value_phi0_twotailed < alpha
    fprintf('Conclusion: Reject null hypothesis that phi_0 = 0 (
        significance level %.2f%%)\n', alpha * 100);
else
    fprintf('Conclusion: Cannot reject null hypothesis that phi_0 = 0 (
        significance level %.2f%%)\n', alpha * 100);
end

fprintf('\nTest for phi_1 > 0:\n');
fprintf('phi_1 mean: %.6f (%.2f%% Confidence Interval: [%.6f, %.6f])\n',
    phi_1_avg, (1-alpha)*100, conf_int_phi1(1), conf_int_phi1(2));
fprintf('Standard error of phi_1: %.6f\n', std_err_phi1);
fprintf('t-statistic: %.4f\n', t_stat_phi1);
fprintf('p-value (one-tailed): %.4f\n', p_value_phi1_onetailed);
if p_value_phi1_onetailed < alpha
    fprintf('Conclusion: Significant evidence that phi_1 > 0 (significance
        level %.2f%%)\n', alpha * 100);
else
    fprintf('Conclusion: No significant evidence that phi_1 > 0 (
        significance level %.2f%%)\n', alpha * 100);
end

%% Data preparation
stock_phi0 = phi_0;
stock_phi1 = phi_1;

stocks_phi_0_avg = phi_0_avg;
stocks_phi_1_avg = phi_1_avg;

stocks_std_err_phi0 = std_err_phi0;
stocks_std_err_phi1 = std_err_phi1;

stocks_t_stat_phi0 = t_stat_phi0;
stocks_t_stat_phi1 = t_stat_phi1;

stocks_p_value_phi0 = p_value_phi0_twotailed;
stocks_p_value_phi1 = p_value_phi1_onetailed;

save("stock_data","stocks_p_value_phi0","stocks_p_value_phi1", ...
    "stocks_t_stat_phi1","stocks_t_stat_phi0","stocks_std_err_phi1", ...
    "stocks_std_err_phi0","stocks_phi_1_avg","stocks_phi_0_avg", ...
    "stock_phi0","stock_phi1")
```

Listing 14: Run E3 Point 2

```
%% Section 2.i
clc
clear
load("cleanData.mat");

% Set analysis period
startDate = datetime('01-Jan-2020');
endDate = datetime('01-Dec-2024');

% Extract Fama-French 6 portfolio returns
ffDates = datetime(cleanData.FF_Portfolios_C{:,1}, 'InputFormat', 'dd-MMM-
    yyyy');
ffReturns = cleanData.FF_Portfolios_C{:,2:7};

% Extract risk-free rates
rfDates = datetime(cleanData.FF_factors_C{:,1}, 'InputFormat', 'dd-MMM-yyyy
    ');
rfRates = cleanData.FF_factors_C{:,5};

% Extract market returns
mktDates = datetime(cleanData.FF_factors_C{:,1}, 'InputFormat', 'dd-MMM-
    yyyy');
mktReturns = cleanData.FF_factors_C{:,2};

% Filter data for analysis period
ffMask = (ffDates >= startDate) & (ffDates <= endDate);
ffReturns = ffReturns(ffMask,:);
ffDates = ffDates(ffMask);

rfMask = (rfDates >= startDate) & (rfDates <= endDate);
rfRates = rfRates(rfMask);

mktMask = (mktDates >= startDate) & (mktDates <= endDate);
mktReturns = mktReturns(mktMask);

% Ensure all data series have the same length
minLength = min([length(rfRates), length(mktReturns), size(ffReturns,1)]);

% Calculate betas for the 6 FF portfolios
num_portfolios = size(ffReturns, 2);
betas = zeros(num_portfolios, 1);
Excess = zeros(minLength, num_portfolios);

for i = 1:num_portfolios
    Market_Excess = mktReturns - rfRates;
    Asset_Excess = ffReturns(:,i) - rfRates;
    Excess(:,i) = Asset_Excess;

    % Regression to estimate beta
```

```
    tbl = table(Market_Excess, Asset_Excess, 'VariableNames', {'
        Market_Excess', 'Asset_Excess'});
    lm = fitlm(tbl, 'Asset_Excess ~ Market_Excess');
    betas(i) = lm.Coefficients.Estimate(2);
end

%% Section 2.ii
% Determine number of years in dataset
years = year(ffDates);
unique_years = unique(years);
num_years = length(unique_years);

% Pre-allocate phi0 and phi1
phi_0 = zeros(num_years, 1);
phi_1 = zeros(num_years, 1);

% Loop through each year
for t = 1:num_years
    % Select observations for current year
    year_idx = years == unique_years(t);

    % Perform cross-section regression for each month in year
    months_in_year = sum(year_idx);
    phi_0_year = zeros(months_in_year, 1);
    phi_1_year = zeros(months_in_year, 1);

    month_counter = 1;
    for m = find(year_idx)'
        % Excess returns for current month
        month_extra_return = Excess(m, :)';

        % Cross-section regression
        tbl_cross = table(betas, month_extra_return, 'VariableNames', {'
            Beta', 'ExcessReturn'});
        mdl = fitlm(tbl_cross, 'ExcessReturn ~ Beta');

        % Extract coefficients
        phi_0_year(month_counter) = mdl.Coefficients.Estimate(1);
        phi_1_year(month_counter) = mdl.Coefficients.Estimate(2);

        month_counter = month_counter + 1;
    end

    % Average coefficients for the year
    phi_0(t) = mean(phi_0_year);
    phi_1(t) = mean(phi_1_year);
end

%% Section 2.iii
% Calculate overall averages of phi_0 and phi_1
```

```
phi_0_avg = mean(phi_0);
phi_1_avg = mean(phi_1);

%% Section 2.iv
% Hypothesis testing for CAPM validity
% H0: phi_0 = 0 and phi_1 > 0

% Calculate standard errors for phi_0 and phi_1 means
std_err_phi0 = std(phi_0) / sqrt(length(phi_0));
std_err_phi1 = std(phi_1) / sqrt(length(phi_1));

% Significance level
alpha = 0.05;
df_phi0 = length(phi_0) - 1;
df_phi1 = length(phi_1) - 1;

% Calculate t-statistic for phi_0 (test for phi_0 = 0)
t_stat_phi0 = phi_0_avg / std_err_phi0;
p_value_phi0_twotailed = 2 * (1 - tcdf(abs(t_stat_phi0), df_phi0));

% Test for phi_1 > 0 (one-tailed)
t_stat_phi1 = phi_1_avg / std_err_phi1;
p_value_phi1_onetailed = 1 - tcdf(t_stat_phi1, df_phi1);

% Display test results
fprintf('\n--- Hypothesis testing for CAPM validity on 6 FF portfolios ---\
    n');
fprintf('Null Hypothesis H0: phi_0 = 0 and phi_1 > 0 (significance level
    %.2f%%)\n\n', alpha * 100);

fprintf('Test for phi_0 = 0:\n');
fprintf('phi_0 mean: %.6f\n', phi_0_avg);
fprintf('Standard error of phi_0: %.6f\n', std_err_phi0);
fprintf('t-statistic: %.4f\n', t_stat_phi0);
fprintf('p-value (two-tailed): %.4f\n', p_value_phi0_twotailed);
if p_value_phi0_twotailed < alpha
    fprintf('Conclusion: Reject null hypothesis that phi_0 = 0\n');
else
    fprintf('Conclusion: Cannot reject null hypothesis that phi_0 = 0\n');
end

fprintf('\nTest for phi_1 > 0:\n');
fprintf('phi_1 mean: %.6f\n', phi_1_avg);
fprintf('Standard error of phi_1: %.6f\n', std_err_phi1);
fprintf('t-statistic: %.4f\n', t_stat_phi1);
fprintf('p-value (one-tailed): %.4f\n', p_value_phi1_onetailed);
if p_value_phi1_onetailed < alpha
    fprintf('Conclusion: Significant evidence that phi_1 > 0\n');
else
    fprintf('Conclusion: No significant evidence that phi_1 > 0\n');
```

```
end

%% Data preparation
ff_phi0 = phi_0;
ff_phi1 = phi_1;

ff_phi_0_avg = phi_0_avg;
ff_phi_1_avg = phi_1_avg;

ff_std_err_phi0 = std_err_phi0;
ff_std_err_phi1 = std_err_phi1;

ff_t_stat_phi0 = t_stat_phi0;
ff_t_stat_phi1 = t_stat_phi1;

ff_p_value_phi0 = p_value_phi0_twotailed;
ff_p_value_phi1 = p_value_phi1_onetailed;

save("ff_data","ff_p_value_phi1","ff_p_value_phi0","ff_t_stat_phi1", ...
    "ff_t_stat_phi0","ff_std_err_phi1","ff_std_err_phi0","ff_phi_0_avg",
        ...
    "ff_phi_1_avg","ff_phi0","ff_phi1")
```

Listing 15: Run E3 Point 3

```
clc
clear
load ff_dati.mat
load stock_dati.mat

% Create a comparison table of main results
comparison_table = table([stocks_phi_0_avg; ff_phi_0_avg], ...
                         [stocks_phi_1_avg; ff_phi_1_avg], ...
                         [stocks_std_err_phi0; ff_std_err_phi0], ...
                         [stocks_std_err_phi1; ff_std_err_phi1], ...
                         [stocks_t_stat_phi0; ff_t_stat_phi0], ...
                         [stocks_t_stat_phi1; ff_t_stat_phi1], ...
                         [stocks_p_value_phi0; ff_p_value_phi0], ...
                         [stocks_p_value_phi1; ff_p_value_phi1], ...
                         'VariableNames', {'Phi0', 'Phi1', 'SE_Phi0', '
                             SE_Phi1', ...
                         'tStat_Phi0', 'tStat_Phi1', 'pValue_Phi0', '
                             pValue_Phi1'}, ...
                         'RowNames', {'Individual_Stocks', 'FF_Portfolios'
                             });

% Display the comparison table
disp('Comparison between individual stocks and FF portfolios results:');
disp(comparison_table);

% Calculate percentage difference in market risk premium
```

```
phi1_diff_pct = (ff_phi_1_avg - stocks_phi_1_avg) / stocks_phi_1_avg * 100;
fprintf('\nPercentage difference in market risk premium (Phi1): %.2f%%\n',
    phi1_diff_pct);

% Test for difference between the two risk premiums
combined_variance = stocks_std_err_phi1^2 + ff_std_err_phi1^2;
t_stat_diff = (ff_phi_1_avg - stocks_phi_1_avg) / sqrt(combined_variance);
df_approx = length(ff_phi0) + length(stock_phi0) - 2; % degrees of freedom
    approximation
p_value_diff = 2 * (1 - tcdf(abs(t_stat_diff), df_approx));

fprintf('\nTest for difference between risk premiums:\n');
fprintf('Difference (Portfolios - Stocks): %.6f\n', ff_phi_1_avg -
    stocks_phi_1_avg);
fprintf('t-statistic: %.4f\n', t_stat_diff);
fprintf('p-value: %.4f\n', p_value_diff);

if p_value_diff < 0.05
    fprintf('Conclusion: The difference is statistically significant at 5%%
        level\n');
else
    fprintf('Conclusion: The difference is not statistically significant at
        5%% level\n');
end
```

Listing 16: Run E4

```
%% Data Import
clear; clc; close all;

% Load the cleaned dataset
load('cleanData.mat','cleanData');
if istable(cleanData)
    T = cleanData;
elseif isfield(cleanData,'FF\_Portfolios\_C')
    T = cleanData.FF\_Portfolios\_C;
else
    error('cleanData.mat: no table found (field "FF\_Portfolios\_C").');
end

% Rename table variables
T.Properties.VariableNames = { ...
    'Date', ...
    'SMALL\_LoBM','ME1\_BM2','SMALL\_HiBM', ...
    'BIG\_LoBM','ME2\_BM2','BIG\_HiBM'};

%% FF3 Factors Calculation
rf      = zeros(height(T),1);              % Risk-free rate set to zero
R\_M    = mean(T{:,2:end},2);               % Average return of all
    portfolios
Mkt\_RF = R\_M - rf;                         % Market excess return
```

```
24
25 small  = mean([T.SMALL\_LoBM T.ME1\_BM2 T.SMALL\_HiBM],2);  % Small
       portfolio average
26 big    = mean([T.BIG\_LoBM   T.ME2\_BM2 T.BIG\_HiBM ],2);   % Big portfolio
       average
27 SMB    = small - big;                     % Size factor
28
29 highBM = 0.5*(T.SMALL\_HiBM + T.BIG\_HiBM); % High book-to-market
30 lowBM  = 0.5*(T.SMALL\_LoBM + T.BIG\_LoBM); % Low book-to-market
31 HML    = highBM - lowBM;                   % Value factor
32
33 X = [Mkt\_RF SMB HML];                     % Factor matrix
34
35 %% FF3 Regressions
36 yNames  = T.Properties.VariableNames(2:end);
37 results = struct();
38
39 fprintf('\n===== FF(3) Regression Results =====\n');
40 for k = 1:numel(yNames)
41     y   = T.(yNames{k}) - rf;  % Excess return for the portfolio
42     mdl = fitlm(X,y,'Intercept',true,'VarNames',{'Mkt\_RF','SMB','HML','
          ExcessRet'});
43     results.(yNames{k}) = mdl;
44
45     c = mdl.Coefficients;
46     fprintf('\n--- %s ---\n',yNames{k});
47     fprintf(' $\alpha$    = %8.4f (SE %6.4f)\n',c.Estimate(1),c.SE(1));
48     fprintf(' $\beta_{M}$  = %8.4f (SE %6.4f)\n',c.Estimate(2),c.SE(2));
49     fprintf(' $\beta_{SMB}$= %8.4f (SE %6.4f)\n',c.Estimate(3),c.SE(3));
50     fprintf(' $\beta_{HML}$= %8.4f (SE %6.4f)\n',c.Estimate(4),c.SE(4));
51     fprintf(' Adj R$^2$ = %6.4f\n',mdl.Rsquared.Adjusted);
52 end
53
54 %% Residual Diagnostics
55 lags          = 24;
56 col\_stem      = [0.4940 0.1840 0.5560];
57 col\_confBound = [0 0.5 0];
58
59 fprintf('\n===== Residual Diagnostics =====\n');
60 for k = 1:numel(yNames)
61     port = yNames{k};
62     res  = results.(port).Residuals.Raw;
63
64     %% ACF (Autocorrelation Function)
65     [acf,l,bounds] = autocorr(res,'NumLags',lags);
66     fig = figure('Visible','off'); hold on;
67     hStem = stem(l,acf,'filled', ...
68         'MarkerFaceColor',col\_stem,'MarkerEdgeColor',col\_stem, ...
69         'LineWidth',1.1,'Color',col\_stem);
70     yline(0,'k');
```

```
    hCB = yline(bounds(1),'Color',col\_confBound,'LineWidth',1.1);
    yline(-bounds(1),'Color',col\_confBound,'LineWidth',1.1);
    grid on; hold off;
    title(['ACF - ' strrep(port,'\_','\\\_')]);
    xlabel('Lag'); ylabel('Sample Autocorrelation');
    legend([hStem hCB],{'ACF','Confidence Bound'},'Location','northeast');
    saveas(fig,['ACF\_' port '.png']); close(fig);

    %% PACF (Partial Autocorrelation Function)
    [pacf,lp,bounds\_p] = parcorr(res,'NumLags',lags);
    fig = figure('Visible','off'); hold on;
    hStem = stem(lp,pacf,'filled', ...
        'MarkerFaceColor',col\_stem,'MarkerEdgeColor',col\_stem, ...
        'LineWidth',1.1,'Color',col\_stem);
    yline(0,'k');
    hCB = yline(bounds\_p(1),'Color',col\_confBound,'LineWidth',1.1);
    yline(-bounds\_p(1),'Color',col\_confBound,'LineWidth',1.1);
    grid on; hold off;
    title(['PACF - ' strrep(port,'\_','\\\_')]);
    xlabel('Lag'); ylabel('Sample Partial Autocorrelation');
    legend([hStem hCB],{'PACF','Confidence Bound'},'Location','northeast');
    saveas(fig,['PACF\_' port '.png']); close(fig);

    %% Residual Tests
    [~,p\_lb6]  = lbqtest(res,'Lags',6);
    [~,p\_lb12] = lbqtest(res,'Lags',12);
    [~,p\_dw]   = dwtest(results.(port));
    [~,p\_arch] = archtest(res,'Lags',1);
    [~,p\_jb]   = jbtest(res);

    fprintf('%-12s | LB(6): %.3f | LB(12): %.3f | DW p: %.3f | ARCH p: %.3f
        | JB p: %.3f\n', ...
        port, p\_lb6, p\_lb12, p\_dw, p\_arch, p\_jb);
end
```

8.2 Functions

Listing 17: Function to Clean Data

```
function cleanData = importCleanData(excelFile)

    sheets = { ...
        'S&P500_TBILL_C', ...
        '100S&PComponents_C', ...
        'FF_Portfolios_C', ...
        'FF_factors_C'   ...
    };

    cleanData = struct();
```

```
    for i = 1:numel(sheets)

        sheet = sheets{i};

        % Import without warnings about names
        opts = detectImportOptions(excelFile, 'Sheet', sheet);
        opts.VariableNamingRule = 'preserve';
        T = readtable(excelFile, opts);

        % Standardize headers (caused problems when referenced later)
        switch sheet
            case 'S&P500_TBILL_C'
                T.Properties.VariableNames = {'Date','SP500','TBILL'};
            otherwise

                % First column = Date, remaining are valid tickers/
                    variables
                T.Properties.VariableNames{1} = 'Date';
                vars = matlab.lang.makeValidName(T.Properties.VariableNames
                    (2:end));
                T.Properties.VariableNames(2:end) = vars;
        end

        % Assign to struct field
        fld = matlab.lang.makeValidName(sheet);
        cleanData.(fld) = T;
    end
end
```

Listing 18: Function to Compute Daily Returns

```
function dailyReturns = computeDailyReturns(cleanData)
    % Compute S&P 500 daily returns
    tabellaImport = cleanData.S_P500_TBILL_C;
    dates = tabellaImport.Date;
    prices = tabellaImport.SP500;
    retSP = prices(2:end) ./ prices(1:end-1) - 1;  % Simple daily returns
        calculation
    tblSP = table(dates(2:end), retSP, 'VariableNames', {'Date','Return'});
    dailyReturns.SP500 = tblSP;

    % Compute component stocks daily returns
    tab100 = cleanData.x100S_PComponents_C;
    dates_b = tab100.Date;
    vars = tab100.Properties.VariableNames(2:end); % Get all stock tickers
    prices_b = tab100{:,vars};
    rendimenti_matrice = prices_b(2:end,:) ./ prices_b(1:end-1,:) - 1; %
        Matrix of daily returns

    % Create table with component returns
```

```
    tickers = matlab.lang.makeValidName(vars); % Ensure valid MATLAB
        variable names
    tblComp = array2table(rendimenti_matrice, 'VariableNames', tickers);
    tblComp.Date = dates_b(2:end);
    tblComp = movevars(tblComp, 'Date', 'Before', 1); % Move date column to
         first position
    dailyReturns.Components = tblComp;
end
```

Listing 19: Function to Analyze S&P500

```
function analysis = analyzeSP500(cleanData, dailyReturns, baseDir)

    % Prepare output directories and figures
    fOutput = fullfile(baseDir,'output');
    fFigure = fullfile(baseDir,'figures');

    % Create figures directory if it doesn't exist
    if ~exist(fFigure, 'dir')
        mkdir(fFigure)
    end

    % Index price table
    dataIndex = cleanData.S_P500_TBILL_C;
    tabPrices = table(dataIndex.Date, dataIndex.SP500, 'VariableNames',{'
        Date','Price'});
    analysis.prices = tabPrices;

    % Return statistics
    tabReturns = dailyReturns.SP500;  % table(Date,Return)
    analysis.stats = computeMoments(tabReturns.Return);

    % ======================
    % 1. PRICE LEVELS PLOT (Modern Area Chart)
    % ======================
    figPrices = figure('Visible','off', 'Color', [1 1 1], 'Position', [100
        100 900 450]);

    ax1 = axes('Parent', figPrices, ...
               'FontName', 'Arial', ...
               'FontSize', 10, ...
               'FontWeight', 'bold', ...
               'XColor', [0.3 0.3 0.3], ...
               'YColor', [0.3 0.3 0.3]);

    % Modern area plot with gradient
    area(tabPrices.Date, tabPrices.Price, ...
        'FaceColor', [0.01 0.66 0.62], ...  % Teal color
        'EdgeColor', [0 0.4 0.4], ...
        'LineWidth', 1.2, ...
        'FaceAlpha', 0.8);
```

```
39
40      % Formatting
41      title('S&P 500 Historical Price Levels', ...
42            'FontSize', 14, 'FontWeight', 'bold', 'Color', [0.2 0.2 0.2]);
43      xlabel('Year', 'FontSize', 12);
44      ylabel('Price (USD)', 'FontSize', 12);
45
46      % Add grid and frame
47      grid on;
48      box on;
49      set(ax1, 'GridAlpha', 0.3, 'Layer', 'top');
50
51      % Set date format
52      datetick('x', 'yyyy', 'keeplimits');
53
54      % Save
55      saveas(figPrices, fullfile(fFigure,'fig_sp500_prices.png'));
56      close(figPrices);
57
58      % ======================
59      % 2. DAILY RETURNS PLOT (Professional Line Chart)
60      % ======================
61      figReturns = figure('Visible','off', 'Color', [1 1 1], 'Position', [100
            100 900 450]);
62
63      ax2 = axes('Parent', figReturns, ...
64                 'FontName', 'Arial', ...
65                 'FontSize', 10, ...
66                 'FontWeight', 'bold', ...
67                 'XColor', [0.3 0.3 0.3], ...
68                 'YColor', [0.3 0.3 0.3]);
69
70      % Enhanced line plot
71      plot(tabReturns.Date, tabReturns.Return, ...
72          'Color', [0.8 0.2 0.2], ...  % Dark red
73          'LineWidth', 1.1);
74
75      % Add zero reference line
76      hold on;
77      yline(0, '--', 'Color', [0.4 0.4 0.4], 'LineWidth', 0.8);
78
79      % Formatting
80      title('S&P 500 Daily Returns', ...
81            'FontSize', 14, 'FontWeight', 'bold', 'Color', [0.2 0.2 0.2]);
82      xlabel('Year', 'FontSize', 12);
83      ylabel('Daily Return (%)', 'FontSize', 12);
84
85      % Grid and frame
86      grid on;
87      box on;
```

```
    set(ax2, 'GridAlpha', 0.2, 'Layer', 'top');

    % Date formatting
    datetick('x', 'yyyy', 'keeplimits');

    % Save
    saveas(figReturns, fullfile(fFigure,'fig_sp500_returns.png'));
    close(figReturns);

    % ======================
    % 3. GARCH CONDITIONAL VARIANCE (Professional Area Chart)
    % ======================
    % GARCH model estimation
    modelloGARCH = garch(1,1);
    [stimaGARCH, ~, ~] = estimate(modelloGARCH, tabReturns.Return, 'Display
        ','off');
    varCondizionata = infer(stimaGARCH, tabReturns.Return);

    % Save results
    analysis.garchModel = stimaGARCH;
    analysis.condVar = varCondizionata;

    % Create figure
    figCondVar = figure('Visible','off', 'Color', [1 1 1], 'Position', [100
         100 900 450]);

    ax3 = axes('Parent', figCondVar, ...
               'FontName', 'Arial', ...
               'FontSize', 10, ...
               'FontWeight', 'bold', ...
               'XColor', [0.3 0.3 0.3], ...
               'YColor', [0.3 0.3 0.3]);

    % Enhanced area plot for volatility
    area(tabReturns.Date(1:numel(varCondizionata)), varCondizionata, ...
        'FaceColor', [0.31 0.4 0.58], ...  % Navy blue
        'EdgeColor', [0.1 0.1 0.3], ...
        'LineWidth', 1.2, ...
        'FaceAlpha', 0.7);

    % Add mean volatility line
    meanVol = mean(varCondizionata);
    hold on;
    yline(meanVol, '--', 'Color', [0.8 0.4 0], 'LineWidth', 1.5, ...
          'Label', sprintf('Mean Volatility: %.4f', meanVol), ...
          'LabelHorizontalAlignment', 'left', ...
          'FontSize', 9);

    % Formatting
    title('S&P 500 Conditional Variance (GARCH(1,1) Model)', ...
```

```
          'FontSize', 14, 'FontWeight', 'bold', 'Color', [0.2 0.2 0.2]);
    xlabel('Year', 'FontSize', 12);
    ylabel('Conditional Variance', 'FontSize', 12);

    % Grid and frame
    grid on;
    box on;
    set(ax3, 'GridAlpha', 0.3, 'Layer', 'top');

    % Date formatting
    datetick('x', 'yyyy', 'keeplimits');

    % Save
    saveas(figCondVar, fullfile(fFigure,'fig_sp500_condvar.png'));
    close(figCondVar);

    % Save all results
    save(fullfile(fOutput,'sp500Analysis.mat'),'analysis');
end
```

Listing 20: Function to Compute Moments

```
function m = computeMoments(ret)
    m = struct();
    m.mean     = mean(ret,'omitnan');
    m.variance = var(ret,'omitnan');
    m.skewness = skewness(ret);
    m.kurtosis = kurtosis(ret);
end
```

Listing 21: Function to Fit Normal

```
function params = fitNormal(ret)
    [muHat, sigmaHat] = normfit(ret);
    params = struct('mu',muHat,'sigma',sigmaHat);
end
```

Listing 22: Function to Perform the Jarque-Bera test

```
function jb = jarqueBeraTest(ret)
    [h,p,jbstat] = jbtest(ret,0.05);
    jb = struct('h',h,'pValue',p,'stat',jbstat);
end
```

Listing 23: Function to Trasform TBILLs

```
function monthlyTbill = transformTBILL(cleanData)

    % Load Date series and annual TBILL
    T = cleanData.S_P500_TBILL_C;
```

```
    dates = T.Date;
    annYield = T.TBILL / 100;

    % Convert annual yield to daily yield
    dailyT = (1 + annYield).^(1/360) - 1;

    % Group by month-year
    y = year(dates);
    m = month(dates);
    grp = findgroups(y, m);

    % monthly label = first day of the month
    monthDate = splitapply(@(d) d(1), dates, grp);

    % cumulative monthly yield
    tbm = splitapply(@(r) prod(1+r)-1, dailyT, grp);

    % Output
    monthlyTbill = table(monthDate, tbm, 'VariableNames',{'Date','
        TBILL_Monthly'});
end
```

Listing 24: Function to Compute Covariance Matrix

```
function C = computeCovMatrix(R)
    C = cov(R,'omitrows');
end
```

Listing 25: Function to Analyze the Portfolio

```
% functions/analyzePortfolio.m

function analyzePortfolio(~, compReturns, tickers, label, varargin)

    % Apply time filter if provided in varargin
    if ~isempty(varargin)
        dateRange = varargin{1};
        dateVec = compReturns.Date;
        mask = dateVec >= dateRange(1) & dateVec <= dateRange(2);
        compReturns = compReturns(mask, :);
    end

    % Select only Date column and desired tickers
    vars = [{'Date'}, tickers]; % cell array with 'Date' + each ticker
    T = compReturns(:, vars); % extract sub-table

    % Extract returns matrix
    R = T{:, 2:end};

    % Initialize output structs
```

```
    fitParams = struct();
    moments = struct();
    jbResults = struct();

    % Loop through each ticker
    for i = 1:numel(tickers)

        tk = tickers{i};  % Current ticker name
        r = R(:, i);      % Returns of i-th stock

        fitParams.(tk) = fitNormal(r);
        moments.(tk) = computeMoments(r);
        jbResults.(tk) = jarqueBeraTest(r);
    end

    % Cross-sectional covariance matrix
    covMatrix = computeCovMatrix(R);

    % Save results
    outDir = fullfile(pwd, 'output');
    save(fullfile(outDir, [label 'Results.mat']),'fitParams', 'moments', '
        jbResults', 'covMatrix');
end
```

Listing 26: Function to Compute Monthly Returns

```
function monthlyReturns = computeMonthlyReturns(dailyReturns)

    %% Period of interest (January 2020 - April 2025)
    startYear = 2020; startMonth = 1; % year month
    endYear = 2025; endMonth = 4; % year month
    periodCodeStart = startYear*100 + startMonth;
    periodCodeEnd = endYear*100 + endMonth;


    %% MONTHLY RETURNS FOR S&P500
    % Select only the desired period
    tblSP = dailyReturns.SP500;
    dateCodesSP = year(tblSP.Date)*100 + month(tblSP.Date);
    maskSP = dateCodesSP >= periodCodeStart & dateCodesSP <= periodCodeEnd;
    tblSP = tblSP(maskSP, :);

    % Group by year and month
    [grpSP, yearsSP, monthsSP] = findgroups(year(tblSP.Date), month(tblSP.
        Date));

    % Calculate monthly cumulative return (without NaN)
    monthlyR_SP = splitapply(@(x) prod(1 + x, 'omitnan') - 1, tblSP.Return,
         grpSP);

    % Create date vector as first day of each month
```

```
    datesSP = datetime(yearsSP, monthsSP, 1);

    % Output table construction
    monthlyReturns.SP500 = table(datesSP, monthlyR_SP, 'VariableNames', {'Date', 'Return'});


    %% MONTHLY RETURNS FOR THE 100 COMPONENTS
    tblComp = dailyReturns.Components;
    dateCodesC = year(tblComp.Date)*100 + month(tblComp.Date);
    maskC = dateCodesC >= periodCodeStart & dateCodesC <= periodCodeEnd;
    tblComp = tblComp(maskC, :);

    % Group by year and month
    [grpC, yearsC, monthsC] = findgroups(year(tblComp.Date), month(tblComp.Date));
    tickers = tblComp.Properties.VariableNames(2:end);
    nGroups = numel(yearsC);
    nTickers = numel(tickers);

    % Preallocation of monthly returns matrix
    monthlyMat = nan(nGroups, nTickers);
    for i = 1:nTickers
        dailyVec = tblComp{:, tickers{i}};
        monthlyMat(:, i) = splitapply(@(x) prod(1 + x, 'omitnan') - 1, dailyVec, grpC);
    end

    % First day of each month dates for components
    datesC = datetime(yearsC, monthsC, 1);

    % Merge with S&P500 dates for alignment
    [commonDates, iaSP, iaC] = intersect(monthlyReturns.SP500.Date, datesC);

    % Create aligned components output table
    tblCompMonthly = array2table(monthlyMat(iaC, :), 'VariableNames', tickers);
    tblCompMonthly.Date = commonDates;
    tblCompMonthly = movevars(tblCompMonthly, 'Date', 'Before', 1);
    monthlyReturns.Components = tblCompMonthly;
end
```

Listing 27: Function to Compute Annual Fama-French Returns

```
function annualReturns = computeAnnualFFReturns(Tff)

    % Extract years and groups
    y = year(Tff.Date);
    [G, years] = findgroups(y);  % G is vector of labels, years is list of years
```

```
6
7       % Prepare output
8       portNames = Tff.Properties.VariableNames(2:end);
9       nYears = numel(years);
10      nPort = numel(portNames);
11      Rcum = zeros(nYears, nPort);
12
13      % Calculate cumulative return for each portfolio and year
14      for i = 1:nPort
15          colPct = Tff{:,portNames{i}};
16          colDec = colPct ./ 100;
17          Rcum(:,i) = splitapply(@(r) prod(1+r)-1, colDec, G);
18      end
19
20      % Build the final table
21      annualReturns = array2table(Rcum, 'VariableNames', portNames);
22      annualReturns.Year = years;
23      annualReturns = movevars(annualReturns, 'Year', 'Before', 1);
24  end
```